TOPIC 3&4:

Content-Based & Multimodal Recommendation

ระบบผู้แนะนำอิงเนื้อหาหลายรูปแบบ

204729
Recommender Systems



ผู้ช่วยศาสตราจารย์ ดร.ดุษฎี ประเสริฐธิติพงษ์

COMPUTER SCIENCE

Content-Based Recommender System



TF-IDF: จาก Synopsis สู่ Sparse Item Vector

คำนวณจริงของ i_1 Toy Story 5 ใน MovieMini-12



1 Preprocess

ทำความสะอาดและเตรียมข้อความ

Original synopsis
มี 48 tokens

↓

Lowercasing, tokenization
และลบ 22 stopword tokens

↓

เหลือ 26 tokens

↓

มี 21 unique terms

ตัวอย่างลำดับคำหลังการทำความสะอาด

cowboy toy space ranger toy lead toys girl ... tablet ... playtime toys ... friendship ... toys ... girl back

ไม่ใช้ stemming: toy และ toys เป็นคนละมิติ

2 Term Frequency

ความถี่ของคำในเอกสาร

$$tf(t,d) = 1 + \log_2 f(t,d) \quad \text{for } f > 0$$

term	$f(t,d)$	$tf(t,d)$
toys	3	2.5850
girl	3	2.5850
toy	2	2.0000
friendship	1	1.0000

ใช้ log base 2 และบวก 1
เพื่อหลีกเลี่ยงคำเป็นศูนย์

3 Inverse Document Frequency

ความสำคัญของคำในคอร์ปัส

$$N = 12$$

$$idf(t) = \ln\left(\frac{N+1}{df(t)+1}\right) + 1$$

term	$df(t)$	$idf(t)$
toys	1	2.8718
girl	2	2.4663
toy	1	2.8718
friendship	2	2.4663

ยิ่งพบในเอกสารน้อย
ยิ่งมีค่า *idf* สูง

4 TF-IDF and L^2 Normalization

คำนำหนักและการทำให้เป็นเวกเตอร์หน่วย

$$w(t,d) = tf(t,d) \times idf(t) \qquad \hat{v} = \frac{w}{\|w\|_2}$$

term	f	tf	df	idf	tf-idf	after L^2
toys	3	2.5850	1	2.8718	7.4235	0.4512
girl	3	2.5850	2	2.4663	6.3754	0.3875
toy	2	2.0000	1	2.8718	5.7436	0.3491
playtime	1	1.0000	1	2.8718	2.8718	0.1745
friendship	1	1.0000	2	2.4663	2.4663	0.1499

คำนวณน้ำหนัก $w(t,d)$ และทำ L^2 normalization
เพื่อให้ได้เวกเตอร์หน่วยความยาว 1

5 Sparse Item Vector

เวกเตอร์ไอเทมแบบ Sparse

ขนาดคลังคำ (Vocabulary) $V = 242$

i_1 มี 21 มิติที่ไม่เป็นศูนย์
และ 221 ศูนย์

ทั้งคอร์ปัสมี 270 คำที่ไม่เป็นศูนย์
จาก 2,904 เซลล์

ตัวอย่างเวกเตอร์ i_1 (ยาว 242 มิติ)



ตัวอย่างการวัดความคล้ายกับ i_2

$$\cos(i_1, i_2) = 0.3875 \times 0.1639 + 0.1499 \times 0.1639 = 0.0881$$

shared terms: girl, friendship

คำที่ซ้ำในเรื่องและพบในเอกสารน้อยได้ค่าน้ำหนักสูง; TF-IDF ไม่เข้าใจคำพ้องหรือบริบท

ค่าจากเอกสาร MovieMini-12 ตาราง 3.14–3.15

Synopsis → Tokens (26 tokens) → *tf* → *idf* → *tf-idf* → L^2 Normalize ($\hat{v}$) → Sparse Item Vector $i_1 \in \mathbb{R}^{242}$ → Similarity & Ranking



BM25: ให้คะแนนความเกี่ยวข้องระหว่าง Query กับ Synopsis

ตัวอย่างคำนวณมาตรฐาน Okapi BM25 กับ i1 Toy Story 5



1 Query และเอกสาร

Query q

"toys girl friendship"

Document d

i1 Toy Story 5
(cleaned synopsis)

ความยาวเอกสาร	$d_l = 26$
ความยาวเฉลี่ย	$avgdl = 24.75$
ขนาดคลังข้อมูล	$N = 12$

ความถี่ของคำในเอกสาร d

toys	$f = 3$
girl	$f = 3$
friendship	$f = 1$

2 พารามิเตอร์และการปรับความยาวเอกสาร

พารามิเตอร์มาตรฐาน

$k_1 = 1.2$ $b = 0.75$

คำนวณค่า B สำหรับเอกสารนี้

$$B = 1 - b + b\left(\frac{d_l}{avgdl}\right)$$

$$= 1 - 0.75 + 0.75\left(\frac{26}{24.75}\right)$$

$$= 0.25 + 0.75(1.0505)$$

$$= 0.25 + 0.7879$$

$$B = 1.0379$$

เอกสารยาวกว่าค่าเฉลี่ยเล็กน้อย
จึงถูกปรับความยาว

3 ค่า IDF ของ BM25

$$idf_{BM25}(t) = \ln\left(1 + \frac{N - df(t) + 0.5}{df(t) + 0.5}\right)$$

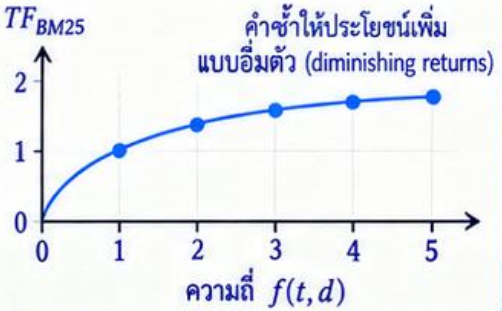
คำ (t)	$df(t)$	ค่า $idf_{BM25}(t)$
toys	1	**2.1595**
girl	2	**1.6487**
friendship	2	**1.6487**

คำที่พบน้อยในคลังข้อมูล
จะได้ค่า IDF สูง

4 ค่า TF แบบอิ่มตัวของ BM25

$$TF_{BM25}(t,d) = \frac{f(t,d)(k_1 + 1)}{f(t,d) + k_1 B}$$

คำ (t)	$f(t,d)$	ค่า $TF_{BM25}(t,d)$
toys	3	**1.5546**
girl	3	**1.5546**
friendship	1	**0.9798**



5 ผลรวมคะแนนความเกี่ยวข้อง

คำ (t)	idf	TF_{BM25}	ผลคูณ (มีส่วนในการคะแนน)
toys	2.1595	1.5546	**3.3571**
girl	1.6487	1.5546	**2.5630**
friendship	1.6487	0.9798	**1.6153**

$$BM25(q, i1) = 3.3571 + 2.5630 + 1.6153$$

$$= 7.5354$$

การจัดลำดับเอกสาร (Ranking)

นำ score(q,d) ของทุกเรื่องมาเรียงจากมากไปน้อย



TF-IDF vs BM25

TF-IDF	BM25
สร้างเวกเตอร์เอกสาร และใช้ cosine	สร้างคะแนนต่อคู่ query-document โดยตรง

สรุปสาระสำคัญ

1. คำที่พบน้อยได้ IDF สูง
2. คำซ้ำให้ประโยชน์เพิ่มแบบอิ่มตัว
3. ความยาวเอกสารถูกปรับด้วย b

หมายเหตุ

ตัวอย่างนี้ใช้พารามิเตอร์มาตรฐาน $k_1 = 1.2$, $b = 0.75$; ค่าคลังและความถี่มาจาก MovieMini-12

จากข้อความ
สู่การค้นพบสิ่งที่ใช่

Better Texts
Brighter People!

Good Texts
Brighter Recommendations
A Kinder Tomorrow

Building a User Profile for Content-Based Recommendation

MOVIEMINI-12 CATALOG (2024–2026)

LEARN FROM WHAT YOU LIKE. FIND WHAT YOU'LL ENJOY NEXT.





1 Collect Feedback (User u_1)

User u_1 rates some movies from MovieMini-12 (2024–2026).

Liked (high ratings)

i1	i5	i8
Dune: Part Two (2024)	Inside Out 2 (2024)	The Wild Robot (2024)
		
★★★★★	★★★★☆	★★★★★
$r_{u1,1} = 5$	$r_{u1,5} = 4$	$r_{u1,8} = 5$

Low-rated (disliked)

i3	i9
Madame Web (2024)	Kravme the Hunter (2024)
	
★☆☆☆☆	★☆☆☆☆
$r_{u1,3} = 1$	$r_{u1,9} = 2$

2 Encode Items in a Common Feature Space

Represent every movie i by a feature vector v_i in the same d-dimensional space (e.g., genres, themes, style).

Movie feature vector v_i

Action · Adventure · Animation · Comedy · Drama · Sci-Fi · … · d dimensions

Example (illustrative features)

	Item	v_i ($d = 6$)
i1	Dune: Part Two (2024)	[0.8, 0.6, 0.0, 0.1, 0.4, 0.7]
i3	Madame Web (2024)	[0.6, 0.2, 0.0, 0.1, 0.2, 0.5]
i5	Inside Out 2 (2024)	[0.1, 0.3, 0.9, 0.8, 0.7, 0.1]
i8	The Wild Robot (2024)	[0.2, 0.5, 0.9, 0.4, 0.6, 0.4]
i9	Kraven the Hunter (2024)	[0.7, 0.4, 0.0, 0.1, 0.3, 0.6]
…		…

All items must be represented in the same vector space (same features, same d).

3 Build the User Profile p_u

Compute the user profile p_u from rated items using one of the following formulas for u_1.

3a Positive-only (liked items)

$$p_u = \text{mean}(v_i \text{ for liked items})$$

$$\text{For } u_1: \quad p_u = \frac{1}{3}(v_1 + v_5 + v_8)$$

3b Rating-centered

$$p_u = \frac{\sum (r_{ui} - \bar{r}_u)\, v_i}{\sum |r_{ui} - \bar{r}_u|}$$

$$\text{For } u_1: \quad p_u = \frac{\sum (r_{u1i} - \bar{r}_{u1})\, v_i}{\sum |r_{u1i} - \bar{r}_{u1}|}$$

where $\bar{r}_u$ is the average rating of u_1's rated items.

3c Rocchio (liked vs. disliked)

$$p_u = \alpha\, \text{mean}(\text{positive}) - \beta\, \text{mean}(\text{negative})$$

$$\text{For } u_1: \quad p_u = \alpha \frac{1}{3}(v_1 + v_5 + v_8) - \beta \frac{1}{2}(v_3 + v_9)$$

Typical choices: $\alpha = 1$, $\beta = 1$ (or tune on validation data).

4 L2 Normalize the Profile

Convert p_u to a unit vector for cosine similarity.

Normalized user profile

$$\hat{p}_u = \frac{p_u}{\|p_u\|_2}$$

$$\text{where} \quad \|p_u\|_2 = \sqrt{\sum_j (p_{u,j})^2}$$

$\hat{p}_{u1}$ is a unit vector ($\|\hat{p}_{u1}\|_2 = 1$) that captures u_1's content preferences in the same feature space as all items.



5 Score, Filter, and Rank

Exclude items u_1 has already rated, score remaining candidates, and rank by similarity.

1 Exclude already-rated items

$$C = \{ i \in \text{catalog} \mid i \notin R_{u1} \}$$

Exclude already-rated items to avoid recommending what the user has already seen.

2 Score candidates

$$s(u, i) = \cos(\hat{p}_u, v_i) = \frac{\hat{p}_u \cdot v_i}{\|\hat{p}_u\|_2 \, \|v_i\|_2}$$

(with $\|\hat{p}_u\|_2 = 1$, this is the cosine similarity)

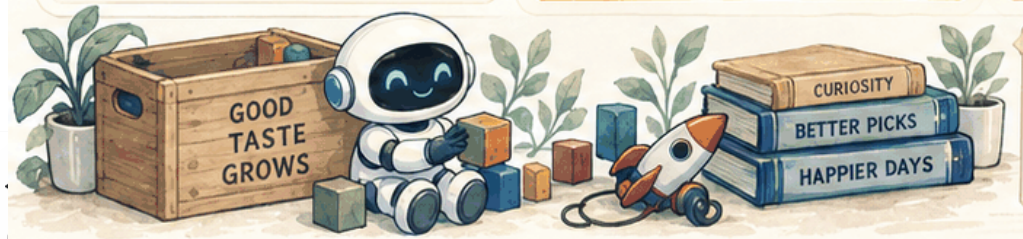
3 Rank and show Top-K

Return the top-K items from C with highest $s(u_1, i)$.

Top-K Candidates for u_1 (example)

Rank	Item	$s(u_1, i)$
1	i11 • Toy Story 5 (2026)	0.82
2	i6 Mufasa (2024)	0.76
3	i12 Zootopia 2 (2025)	0.71
…	…	…

i11 • Toy Story 5 (2026) is a candidate (not a guaranteed recommendation). Final results depend on the full catalog and actual feature vectors.



New feedback arrives …



i10 Wicked (2024) ★★★★★ $r_{u1,10} = 4$

Update profile: $p_u \leftarrow$ recompute (with new ratings)

More feedback.
A clearer profile.
More of what you love.



CNN Encoder

Text CNN Encoder

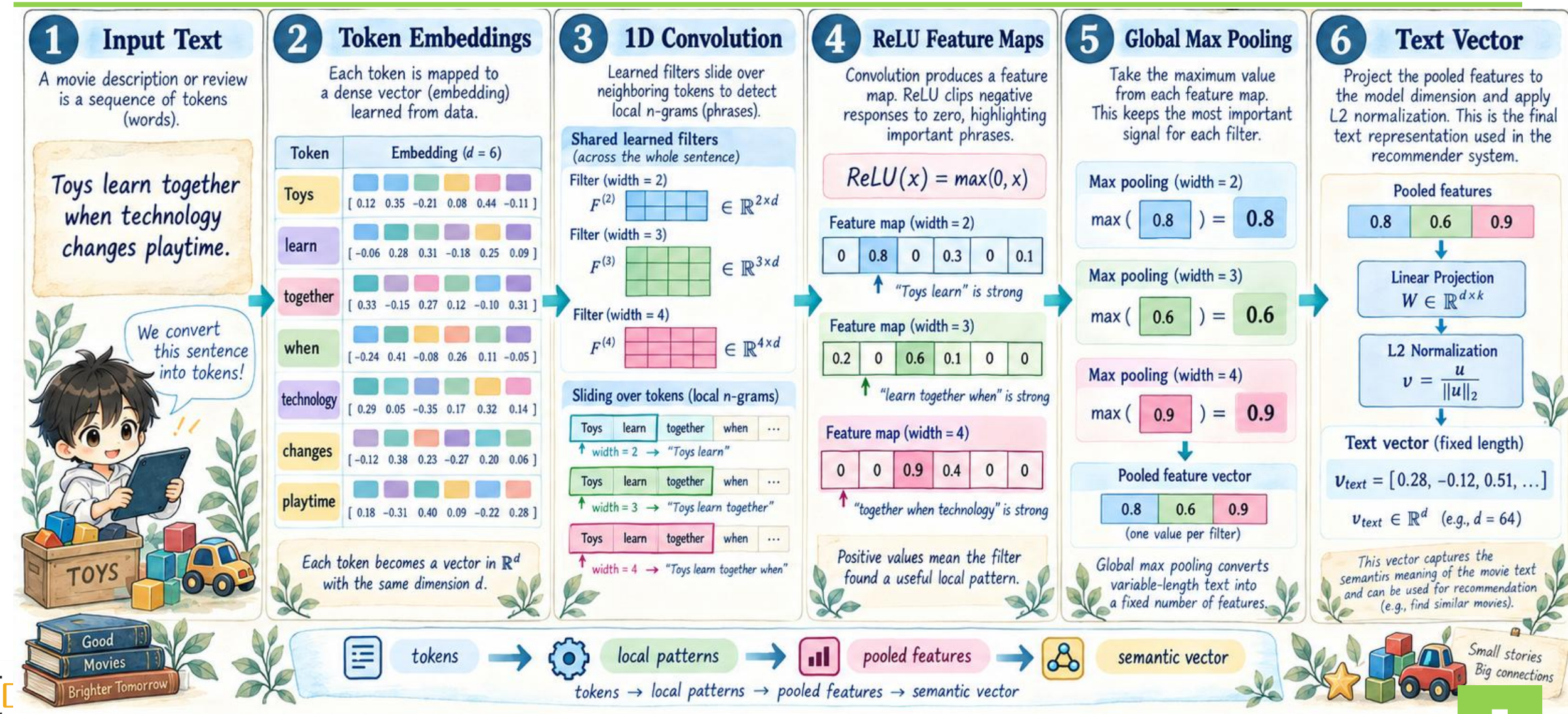


Image CNN Encoder

1 Input Image

A movie image (e.g., a scene or key frame) is a 2D grid of pixels with 3 color channels (R, G, B).



Input image $I \in \mathbb{R}^{H \times W \times 3}$
(e.g., 224 × 224 × 3)

Pixel values (0 – 255)

3 channels: R G B

An image is just numbers, but it contains a rich story!

2 2D Convolution

A learned 3 × 3 kernel slides over the image, computing a dot product at each location to produce a feature map.

Learned 3 × 3 kernel
(shared across the image)

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

(e.g., edge detector)

Input patch (3 × 3 × 3) * →

Convolution result (one position)

	2.1	

stride = 1, padding = 1

The same kernel is applied across the whole image to create a feature map.

3 ReLU Feature Maps

Convolution produces feature maps that respond to patterns (edges, colors, textures). ReLU sets negative values to zero.

$$ReLU(x) = max(0, x)$$

Edge response

-0.6	1.2	-0.3	ReLU →	0	1.2	0
0.8	-1.1	0.5		0.8	0	0.5
-0.4	0.2	1.6		0	0.2	1.6

Color response

-0.3	0.5	-0.1	ReLU →	0	0.5	0
1.2	0.8	-0.4		1.2	0.8	0
-0.2	-0.6	0.7		0	0	0.7

Texture response

-1.0	0.1	0.4	ReLU →	0	0.1	0.4
0.7	-0.5	0.3		0.7	0	0.3
0.2	0.6	-0.1		0.2	0.6	0

Each feature map highlights a different visual pattern (edges, colors, textures, ...).

4 Max Pooling

Max pooling reduces the spatial size (width and height) while keeping the strongest response in each region.

2 × 2 Max Pooling
(stride = 2)

Input feature map (4 × 4)

0.1	0.8	0.3	0.2
0.6	0.4	0.1	0.0
0.2	0.9	0.7	0.1
0.0	0.3	0.2	0.5

2 × 2 windows (non-overlapping)

0.1	0.8	0.3	0.2
0.6	0.4	0.1	0.0
0.2	0.9	0.7	0.1
0.0	0.3	0.2	0.5

max = 0.8 | max = 0.3 | max = 0.9 | max = 0.7

Pooled feature map (2 × 2)

0.8	0.3
0.9	0.7

Pooling keeps the most important signal and makes the model more robust to small shifts.

5 Deep CNN Blocks

Repeat Conv + ReLU + Pooling multiple times. Deeper layers see larger receptive fields and capture higher-level patterns.

Block 1: Conv + ReLU + Pool
detects edges and simple colors
receptive field: 3 × 3

Block 2: Conv + ReLU + Pool
detects textures and parts (e.g., corners, wheels)
receptive field: 5 × 5

Block 3: Conv + ReLU + Pool
detects shapes (e.g., toys, rocket)
receptive field: 9 × 9

Block 4: Conv + ReLU + Pool
detects object-level patterns (e.g., toy box, robot)
receptive field: 15 × 15

Deeper layers capture more semantic information about the image.

6 Global Average Pooling

Each final feature map is averaged over its spatial dimensions to produce one number per channel.

Global Average Pooling (GAP)

$$g_k = \frac{1}{H \times W} \sum_{i=1}^{H} \sum_{j=1}^{W} F_k(i,j)$$

Feature maps (e.g., 3 channels)

→ 0.42
→ −0.08
→ 0.73
... (more channels)

GAP converts each feature map to a single number, making the representation independent of image size.

7 Image Vector

The pooled features are optionally projected to a target dimension and L2 normalized to get a fixed-length vector for recommendation.

Linear Projection

$$z = Wg + b$$
$$z \in \mathbb{R}^d$$

L2 Normalization

$$v_{image} = \frac{z}{\|z\|_2}$$
$$v_{image} \in \mathbb{R}^d$$

Final image vector
(fixed length)

$v_{image} = [0.42, -0.08, 0.73, \ldots]$
(e.g., $d = 64, 128, \ldots$)

This fixed-length vector captures the semantic meaning of the movie image and can be used for recommendation (e.g., find similar movies).



From pixels to meaning!

pixels → local visual patterns → feature maps → pooled features → semantic vector

Same idea, brighter recommendations!

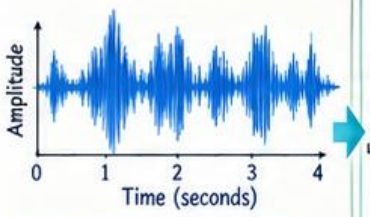


Audio CNN Encoder

1 Input Audio

A movie soundtrack or background audio is a 1D waveform (amplitude over time).

Toy Story 5 (2026) Soundtrack Clip (Example)

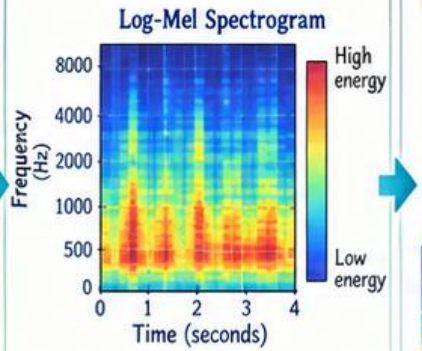


The audio contains music, sound effects and dialogue.



2 Log-Mel Spectrogram

Convert the waveform to a time-frequency representation using Short-Time Fourier Transform (STFT) and Mel filter banks, then apply log scaling.



Vertical axis: frequency (mel scale)
Horizontal axis: time
Color: log energy (louder = warmer)



We convert audio into an image-like spectrogram so CNNs can find patterns!

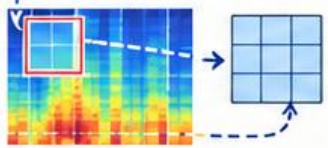
3 2D Convolution

Learned filters (kernels) slide over local time-frequency patches to detect short sound patterns (e.g., drum hits, notes, voice fragments).

A learned kernel (e.g., 3 × 3)

1	0	-1
1	0	-1
1	0	-1

Sliding over spectrogram



Convolve (element-wise multiplication and sum)

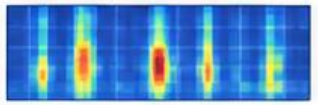
Each filter looks for a specific local sound pattern such as an onset, a harmonic pattern or a frequency sweep.

4 ReLU Feature Maps

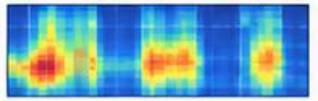
Convolution produces feature maps. ReLU sets negative values to zero, highlighting the presence of detected patterns.

$$ReLU(x) = max(0, x)$$

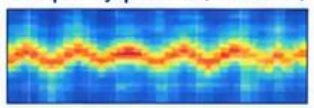
Rhythm (percussive)



Timbre (instrument color)



Frequency pattern (harmonics)

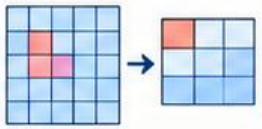


Positive values indicate how strongly the pattern appears at each time and frequency.

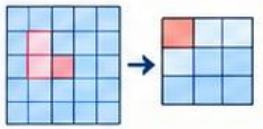
5 Pooling

Pooling reduces the time-frequency resolution while keeping the most important activations. It makes the representation more robust to small shifts in time and frequency.

Max Pooling (2 × 2)



Average Pooling (2 × 2)



Pooling keeps the strongest responses and reduces the size of the feature map (e.g., from $T \times F$ to $T/2 \times F/2$).

6 Deep CNN + GAP

Repeat Conv + ReLU + Pooling for several layers to learn higher-level audio patterns. Then use Global Average Pooling (GAP) to get one number per final feature map.

Log-Mel Spectrogram ($T \times F \times 1$)

↓

Conv + ReLU

Pooling

Conv + ReLU

Pooling

...

Final Feature Maps ($T' \times F' \times C$)

↓

Global Average Pooling (avg over time & frequency)



1 number per channel ($1 \times C$)

7 Audio Vector

The GAP output is a fixed-length vector. We apply a linear projection and L2 normalization to get the final audio embedding used in the recommender system.

Linear Projection

$$z = Wv + b$$
$$z \in \mathbb{R}^d$$

↓

L2 Normalization

$$v_{audio} = z / \|z\|_2$$

↓

Audio Vector (fixed length)

$v_{audio} = [-0.21, 0.37, 0.11, \ldots]$

$v_{audio} \in \mathbb{R}^d$ (e.g., $d = 64$)

This vector captures the semantic meaning of the audio (e.g., mood, genre, instruments) and can be used for recommendation (e.g., find movies with similar soundtracks).

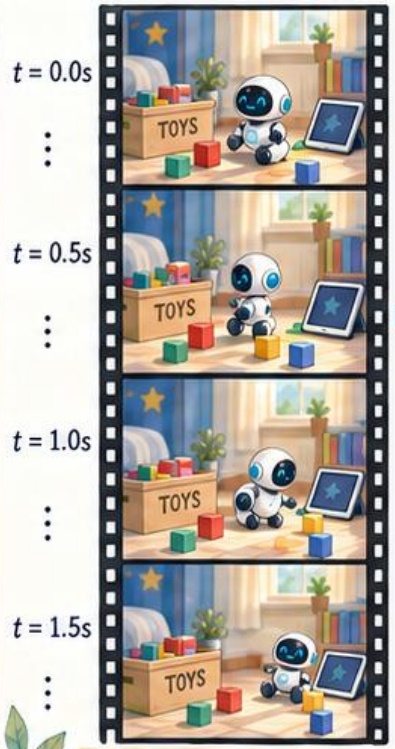




Video CNN Encoder

1 Input Video

A movie is a sequence of frames (images) over time.

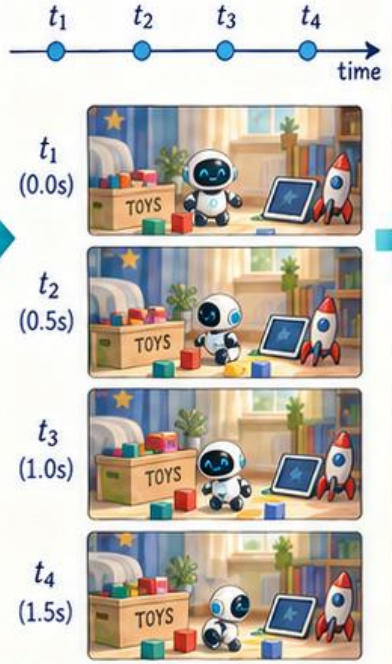


A short video clip from a toy room (showing a moving scene). It contains motion, objects and context over time.

2 Frame Sampling

Select T frames evenly spaced from the video.

Sample $T = 4$ frames

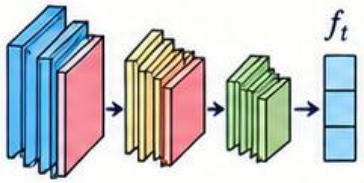


Using evenly spaced frames captures the content and motion in the clip.

3 Shared 2D CNN

Apply the same image CNN (with shared weights) to every sampled frame.

Image CNN (shared weights)



The same CNN weights are used for all frames (time-invariant).

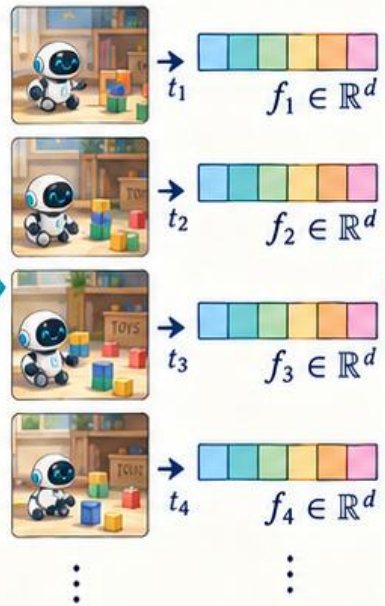
Example CNN (per frame)

Input: 224 × 224 × 3
Conv (3 × 3) + ReLU
Max Pooling (2 × 2)
⋯ (more layers)

Output: $f_t \in \mathbb{R}^d$ (e.g., $d = 512$)

4 Frame Vectors

Each frame is converted to a feature vector.



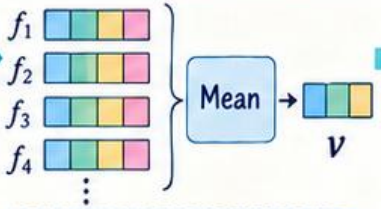
Each frame vector f_t captures what is in the frame (objects, scene, appearance).

5 Temporal Pooling

Aggregate frame vectors over time to get a single video representation.

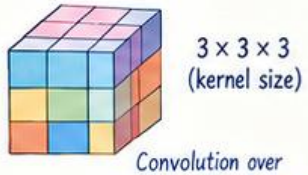
Average Pooling (mean)

$$v = \frac{1}{T}\sum_{t=1}^{T} f_t$$



Alternative: 3D CNN

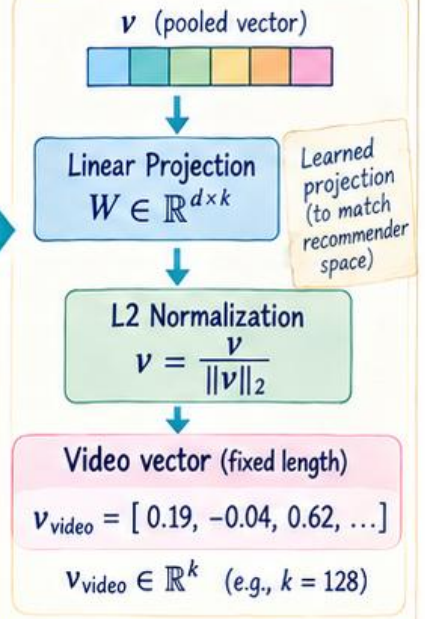
A 3D CNN learns local space-time motion using 3 × 3 × 3 kernels.



Convolution over space (x, y) and time (t)

6 Video Vector

Optionally project the pooled vector and apply L2 normalization to get the final fixed-length video representation.



Video vector (fixed length)

$v_{\text{video}} = [0.19, -0.04, 0.62, \ldots]$

$v_{\text{video}} \in \mathbb{R}^k$ (e.g., $k = 128$)

This fixed-length vector represents the entire video clip and can be used in the recommender system (e.g., to find similar movies).

frames → shared CNN → frame features → temporal pooling → semantic vector

Small videos Big

Text Transformation Encoder

Text Transformation Encoder

1 ข้อความนำเข้า (Input Text)

ประโยคข้อความที่เราต้องการให้ Encoder เข้าใจความหมาย

"Toys learn to work together when new technology changes playtime."



2 ตัดเป็นหน่วยย่อย (Tokenization)

แบ่งประโยคเป็นคำหรือหน่วยย่อย (Token / Subword)

toys	learn	work	together	technology	changes	playtime
101	205	317	450	612	726	893

Token IDs (ตัวเลขแทนคำ)

3 เวกเตอร์ของแต่ละคำ (Token Embedding)

แปลงแต่ละ Token ให้เป็นเวกเตอร์ตัวเลข (Embedding)

toys	0.12	-0.03	0.48	...
learn	-0.26	0.57	0.11	...
work	0.33	0.08	-0.41	...
together	-0.19	0.45	0.27	...
technology	0.41	-0.21	0.63	...
changes	-0.37	0.29	0.18	...
playtime	0.56	-0.12	0.39	...

(เวกเตอร์ที่มีหลายมิติ เช่น 768 มิติ)

4 เพิ่มข้อมูลตำแหน่ง (Positional Encoding)

ใส่ข้อมูลตำแหน่งของคำในประโยค เพื่อให้รู้ลำดับคำ

toys	ตำแหน่งที่ 1 → ...
learn	ตำแหน่งที่ 2 → ...
work	ตำแหน่งที่ 3 → ...
together	ตำแหน่งที่ 4 → ...
technology	ตำแหน่งที่ 5 → ...
changes	ตำแหน่งที่ 6 → ...
playtime	ตำแหน่งที่ 7 → ...

5 Self-Attention มองหาความสัมพันธ์ระหว่างคำ

แต่ละคำจะ "มอง" คำอื่น ๆ ในประโยค เพื่อเข้าใจความหมายจากบริบททั้งหมด



6 Transformer Encoder Blocks

ผ่านขั้นของ Encoder หลายขั้น (หลายรอบ) เพื่อเรียนรู้ความสัมพันธ์ที่ซับซ้อนขึ้น



7 ความหมายจากบริบท (Contextual Meaning)

หลังจากผ่าน Encoder แต่ละคำจะมีเวกเตอร์ที่เข้าใจบริบทจากทุกคำในประโยค

toys	...
learn	...
work	...
together	...
technology	...
changes	...
playtime	...

เวกเตอร์ของแต่ละคำถูกปรับให้สะท้อนความหมายจากบริบท

8 เวกเตอร์ความหมายของประโยค (Sentence Semantic Vector)

นำเวกเตอร์ของทุกคำมารวมกัน (เช่น mean pooling หรือใช้ [CLS]) ได้เป็นเวกเตอร์ความหมายของทั้งประโยค

0.23	-0.11	0.56	0.08	...

(เช่น ขนาด 768 มิติ)

ความหมายโดยรวมของประโยคนี้ คือ...
"animation, friendship, technology, changing playtime"



Text Transformation Encoder : Patching

1 ข้อความเข้า (Input Text)

"Toys learn to work together when new technology changes playtime."



เอกสารข้อความ (Synopsis)

เริ่มจากข้อความยาว ที่ต้องการแนะนำเนื้อหา

2 Tokenization

1. toys
2. learn
3. to
4. work
5. together
6. when
7. new
8. technology
9. changes
10. playtime

$L = 10$ tokens

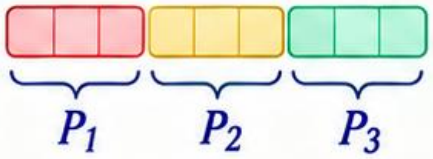
แยกข้อความเป็น Token (หน่วยคำ)

3 กำหนดขนาด Patch

Patch size $P = 4$

Stride $S = 4$

Non-overlapping



แบ่งต่อเนื่องทีละช่ว โดยไม่ซ้ำกัน

4 แบ่งเป็น Text Patches

P_1 = [toys, learn, to, work]

P_2 = [together, when, new, technology]

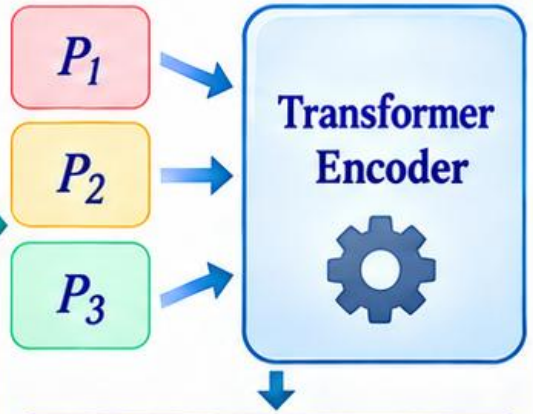
P_3 = [changes, playtime, [PAD], [PAD]]

Attention mask = [1, 1, 0, 0]

$N = \lceil L/P \rceil = \lceil 10/4 \rceil = 3$ patches

ได้ทั้งหมด 3 ช่วง (Patch) โดยช่วงสุดท้ายมีการเติม [PAD]

5 ผลลัพธ์พร้อมเข้ารหัส



1 Patch = 1 หน่วยบริบท

นำแต่ละ Patch ไปสร้างเวกเตอร์ แล้วป้อนเข้า Transformer เพื่อใช้ในการแนะนำเนื้อหา



Text → Tokens → Fixed-size Patches → Patch Embedding

หมายเหตุ:
Transformer ข้อความทั่วไปใช้ Token เป็นหน่วยพื้นฐาน; Text Patching ใช้จัดกลุ่ม Token เป็น Chunk/Window เมื่อต้องประมวลผลข้อความยาว

Text Transformation Encoder : Patch Embedding

1 Text Patch

นำ Token ที่อยู่ติดกัน
มารวมเป็นกลุ่ม (Patch)

$$P_1 = [\text{toys, learn, to, work}]$$

toys | learn | to | work

Patch
กลุ่มของ Token ที่อยู่ติดกัน
ในข้อความ

2 Token Embeddings (2 มิติ)

เวกเตอร์แทนความหมาย
ของแต่ละ Token

Token	Embedding (2 มิติ)
toys	[0.80, 0.20]
learn	[0.30, 0.70]
to	[0.10, 0.10]
work	[0.60, 0.40]

แต่ละ Token ถูกแปลงเป็นเวกเตอร์
ด้วยเมทริกซ์การฝัง
(Embedding Matrix)

3 รวม Token ภายใน Patch

ใช้การหาค่าเฉลี่ย (Mean Pooling)
ของเวกเตอร์ทุก Token ใน Patch

$$m_1 = \frac{1}{4}\sum_i x_i$$

แทนค่าและคำนวณทีละมิติ

$$m_1 = \left[\frac{0.80+0.30+0.10+0.60}{4}, \frac{0.20+0.70+0.10+0.40}{4}\right]$$

$$m_1 = [0.45, 0.35]$$

ได้เวกเตอร์สรุปความหมาย
ของทั้ง Patch

4 Linear Projection

แปลงเวกเตอร์ให้เป็น
ตัวแทนในมิติของโมเดล

$$q_1 = m_1 W + b$$

โดยมี

$$W = \begin{bmatrix} 1.00 & 0.50 \\ -0.50 & 1.00 \end{bmatrix}$$

$$b = [0.10, 0.05]$$

แทนค่าและคำนวณ

$$q_1 = [0.375, 0.625]$$

ได้เวกเตอร์ตัวแทนของ Patch
ในมิติใหม่

5 เพิ่มตำแหน่ง Patch

บวกเวกเตอร์ตำแหน่ง
เพื่อบอกลำดับของ Patch

$$p_1 = [0.020, -0.020]$$

$$e_1 = q_1 + p_1$$

แทนค่าและคำนวณ

$$e_1 = [0.395, 0.605]$$

ได้เวกเตอร์ที่มีข้อมูลเนื้อหา
และตำแหน่งของ Patch

6 Patch Embedding Sequence

ได้ลำดับเวกเตอร์ของทุก Patch
สำหรับนำเข้า Transformer

$$E = \begin{bmatrix} e_1 \\ e_2 \\ e_3 \end{bmatrix}$$

$e_1 = [0.395, 0.605]$

$e_2 = [0.115, 0.780]$

$e_3 = [0.740, 0.610]$

Shape = 3 × 2

Transformer Encoder



Patch Embedding ต่างจาก Token Embedding: หนึ่งเวกเตอร์แทนกลุ่ม Token ทั้ง Patch

Token Embeddings → Patch Pooling → Projection → Positional Embedding → Transformer

หมายเหตุ:
ตัวเลข 2 มิติใช้เพื่อสาธิต;
โมเดลจริงอาจใช้ $d = 768$



Projection + L^2 Normalization: สู่ Item Text Vector

ตัวอย่างย่อจาก Pooled Vector ของ Synopsis เรื่อง Toy Story 5

ข้อความ.... สามารถเปลี่ยนเป็นตัวเลข และตัวเลขสามารถสร้างคำแนะนำที่มีความหมายได้



1 Input pooled vector

ผลจาก Mean Pooling ของ 4 token

$$z = [0.225,\ 0.375,\ 0.300,\ 0.275]$$

0.225	0.375	0.300	0.275

z เป็นตัวแทนข้อความ (4 มิติ) ที่ได้จากการทำ Mean Pooling ของ token ทั้ง 4 ตัว

2 Linear Projection (4 → 2 dimensions)

$$q = Wz + b$$

W (2×4)

$$\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

b (2×1)

$$\begin{bmatrix} 0.05 \\ -0.05 \end{bmatrix}$$

คำนวณที่ละมิติ

$$q_1 = 0.225 + 0.300 + 0.05 = 0.575$$
$$q_2 = 0.375 + 0.275 - 0.05 = 0.600$$

ดังนั้น

$$q = [0.575,\ 0.600]$$

3 Vector length (ขนาดของเวกเตอร์)

$$\|q\|_2 = \sqrt{0.575^2 + 0.600^2} = \sqrt{0.690625} = 0.8310$$

ความยาวของเวกเตอร์ q



ค่านี้บอกขนาด (magnitude) ของเวกเตอร์ก่อนทำ normalization

4 L^2 Normalization (ทำให้มีความยาวเป็น 1)

$$v_i = \frac{q}{\|q\|_2} = \left[\frac{0.575}{0.8310},\ \frac{0.600}{0.8310}\right] = [0.6919,\ 0.7220]$$

ทำให้เวกเตอร์มีความยาวเป็น 1



ยังคงทิศทางเดิม แต่ปรับขนาดให้เป็นเวกเตอร์หน่วย (unit vector)

5 Output Item Text Vector v_i

$$v_i = [0.6919,\ 0.7220]$$

ตรวจสอบความยาว (ควรเป็น 1)

$$\|v_i\|_2 = \sqrt{0.6919^2 + 0.7220^2} \approx 1.0000$$

$$\text{cosine}(p_u, v_i) \rightarrow \text{recommendation score}$$

นำไปเปรียบเทียบกับเวกเตอร์ผู้ใช้ เพื่อจัดอันดับไอเท็มที่เหมาะสม

สรุปแนวคิดสำคัญ

“ Projection ปรับมิติให้ตรงกับปริภูมิระบบแนะนำ; L^2 ทำให้เปรียบเทียบทิศทางด้วย cosine ได้โดยไม่ขึ้นกับขนาด ”

ตัวเลขจำลอง 4 → 2 มิติเพื่อสาธิต; แบบจำลองจริงอาจฉายเป็น 128, 256 หรือ 768 มิติ

จากข้อความสู่ตัวเลข
จากตัวเลขสู่โอกาสที่ดีกว่า

Text Transformation Encoder : Self-Attention

เปลี่ยน token แต่ละตัวให้เข้าใจบริบททั้งประโยค

1 สร้าง Q, K, V

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

ตัวอย่างย่อกำหนด

$$W_Q = W_K = W_V = I$$

และ $d_k = 2$

Token	x_i	Q_i	K_i	V_i
toys	[1.00, 0.00]	[1.00, 0.00]	[1.00, 0.00]	[1.00, 0.00]
tablet	[0.00, 1.00]	[0.00, 1.00]	[0.00, 1.00]	[0.00, 1.00]
playtime	[1.00, 1.00]	[1.00, 1.00]	[1.00, 1.00]	[1.00, 1.00]
friendship	[1.00, 0.50]	[1.00, 0.50]	[1.00, 0.50]	[1.00, 0.50]

เนื่องจากใช้เมทริกซ์เอกลักษณ์ ดังนั้น $Q = K = V = X$

2 คำนวณคะแนนดิบ (Scaled Scores)

$$score(q, k_j) = \frac{q \cdot k_j}{\sqrt{2}}$$

พิจารณา Query ของ token “playtime”

$$q_{,\ playtime} = [1.00, 1.00]$$

Token	k_j	score
toys	[1.00, 0.00]	0.7071
tablet	[0.00, 1.00]	0.7071
playtime	[1.00, 1.00]	1.4142
friendship	[1.00, 0.50]	1.0607

$\sqrt{2} = 1.4142$ (ปัดทศนิยม 4 ตำแหน่ง)

3 คำนวณน้ำหนัก (Softmax Attention)

$$\alpha_j = \frac{\exp(score_j)}{\sum_i \exp(score_i)}$$

Token	score	α_j
toys	0.7071	0.1834
tablet	0.7071	0.1834
playtime	1.4142	0.3720
friendship	1.0607	0.2612

$$\sum_j \alpha_j = 1.0000$$

4 คูณค่าน้ำหนักกับ V (Weighted Values)

$$\alpha_j V_j$$

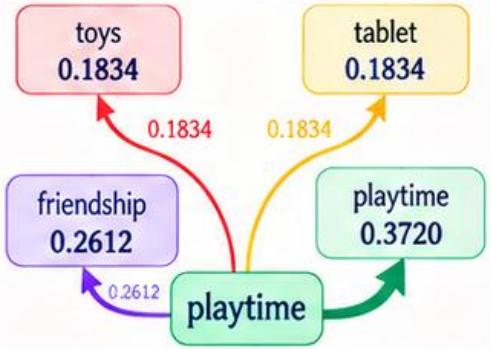
Token	α_j	V_j	$\alpha_j V_j$
toys	0.1834	[1.00, 0.00]	[0.1834, 0.0000]
tablet	0.1834	[0.00, 1.00]	[0.0000, 0.1834]
playtime	0.3720	[1.00, 1.00]	[0.3720, 0.3720]
friendship	0.2612	[1.00, 0.50]	[0.2612, 0.1306]

5 Context Vector

$$h_{playtime} = \sum_j \alpha_j V_j$$

$$h_{playtime} = [0.8166, 0.6860]$$

ภาพรวมการให้ความสนใจจาก token “playtime”



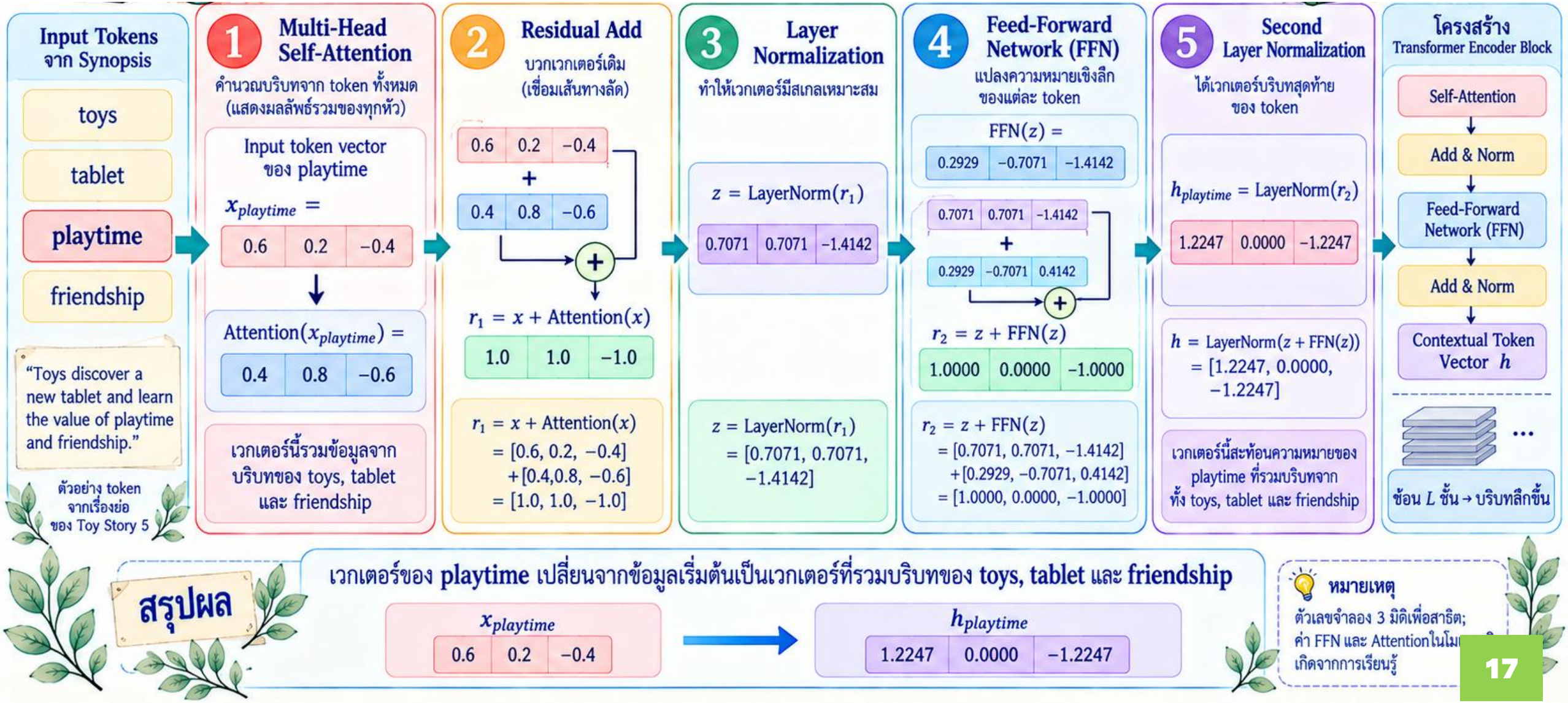
ความหนาของลูกศรแสดงค่าน้ำหนัก (ยิ่งหนายิ่งให้ความสนใจมาก)

สรุปความหมาย → **playtime** รวมข้อมูลของตนเองกับ **toys**, **tablet** และ **friendship** ตามน้ำหนักความเกี่ยวข้อง

ตัวเลขจำลอง 2 มิติเพื่อสาธิตกลไก; โมเดลจริงเรียน W_Q, W_K, W_V และใช้หลาย Attention Heads

Text Transformation Encoder : Transformer Encoder Block

เปลี่ยน token แต่ละตัวให้เข้าใจบริบททั้งประโยค



สรุปผล

เวกเตอร์ของ **playtime** เปลี่ยนจากข้อมูลเริ่มต้นเป็นเวกเตอร์ที่รวมบริบทของ **toys, tablet** และ **friendship**

$x_{playtime}$ 0.6 0.2 −0.4 → $h_{playtime}$ 1.2247 0.0000 −1.2247

หมายเหตุ
ตัวเลขจำลอง 3 มิติเพื่อสาธิต;
ค่า FFN และ Attentionในโม
เกิดจากการเรียนรู้

Text Transformation Encoder : Pooling

Contextual Token Vectors จาก Synopsis ของ **Toy Story 5** (ตัวอย่างบางคำ)

toys	tablet	playtime	friendship
$h_1 = [0.1, 0.4, 0.2, 0.3]$	$h_2 = [0.5, 0.2, 0.1, 0.0]$	$h_3 = [0.3, 0.8, 0.6, 0.1]$	$h_4 = [0.0, 0.1, 0.3, 0.7]$

1 Mean Pooling
หาค่าเฉลี่ยของทุก token

token	$h_i = [h_{i1}, h_{i2}, h_{i3}, h_{i4}]$
toys	[0.1, 0.4, 0.2, 0.3]
tablet	[0.5, 0.2, 0.1, 0.0]
playtime	[0.3, 0.8, 0.6, 0.1]
friendship	[0.0, 0.1, 0.3, 0.7]

$$v_{mean} = \frac{1}{4}\sum_{i=1}^{4} h_i$$
$$= [0.225, 0.375, 0.300, 0.275]$$

ทุก token มีส่วนเท่ากัน

2 Max Pooling
เลือกค่ามากที่สุดในแต่ละมิติ

token	$h_i = [h_{i1}, h_{i2}, h_{i3}, h_{i4}]$
toys	[0.1, 0.4, 0.2, 0.3]
tablet	**0.5** 0.2 0.1 0.0]
playtime	[0.3 **0.8** **0.6** 0.1]
friendship	[0.0, 0.1 0.3 **0.7**

$$v_{max}[j] = \max_i h_i[j]$$
$$= [0.5, 0.8, 0.6, 0.7]$$

เก็บคุณลักษณะที่เด่นที่สุดรายมิติ

3 [CLS] Pooling
ใช้เวกเตอร์พิเศษที่เรียนรู้ภาพรวม

token	$h_i = [h_{i1}, h_{i2}, h_{i3}, h_{i4}]$
[CLS]	$h_{[CLS]} = [0.6, 0.1, 0.1, 0.2]$

$$v_{cls} = h_{[CLS]}$$
$$= [0.6, 0.1, 0.1, 0.2]$$

ใช้เวกเตอร์พิเศษที่เรียนรู้ภาพรวม

4 Weighted Pooling
ถ่วงน้ำหนักตามความสำคัญของ token

token	$h_i = [h_{i1}, h_{i2}, h_{i3}, h_{i4}]$	α_i
toys	[0.1, 0.4, 0.2, 0.3]	0.2
tablet	[0.5, 0.2, 0.1, 0.0]	0.1
playtime	[0.3, 0.8, 0.6, 0.1]	**0.5**
friendship	[0.0, 0.1, 0.3, 0.7]	0.2

$\sum \alpha_i = 1.0$

$$v_{weighted} = \sum_{i=1}^{4} \alpha_i h_i$$
$$= [0.22, 0.52, 0.41, 0.25]$$

ให้ playtime น้ำหนักสูงสุด 0.5

สรุปข้อดีของแต่ละวิธี

Mean	Max	[CLS]	Weighted
เสถียรและง่าย	เน้นสัญญาณเด่น	ขึ้นกับการฝึกโมเดล	เน้น token สำคัญ

ภาพรวมกระบวนการ

หลาย token vectors H → **Pooling** (Mean / Max / [CLS] / Weighted) → เวกเตอร์เดียว z → **Projection** (เช่น $Wz + b$)

ตัวเลขจำลอง 4 มิติเพื่อสาธิต; เอกสาร MovieMini-12 ใช้ Weighted Pooling ถ่วง IDF สำหรับเวกเตอร์ที่รายงาน

Learning Today A Brighter Tomorrow

ข้อความเล็ก ๆ สร้างการเปลี่ยน...ได้เสมอ

COMPUTER SCIENCE

Image Transformation Encoder

ภาพรวม Image Transformation Encoder

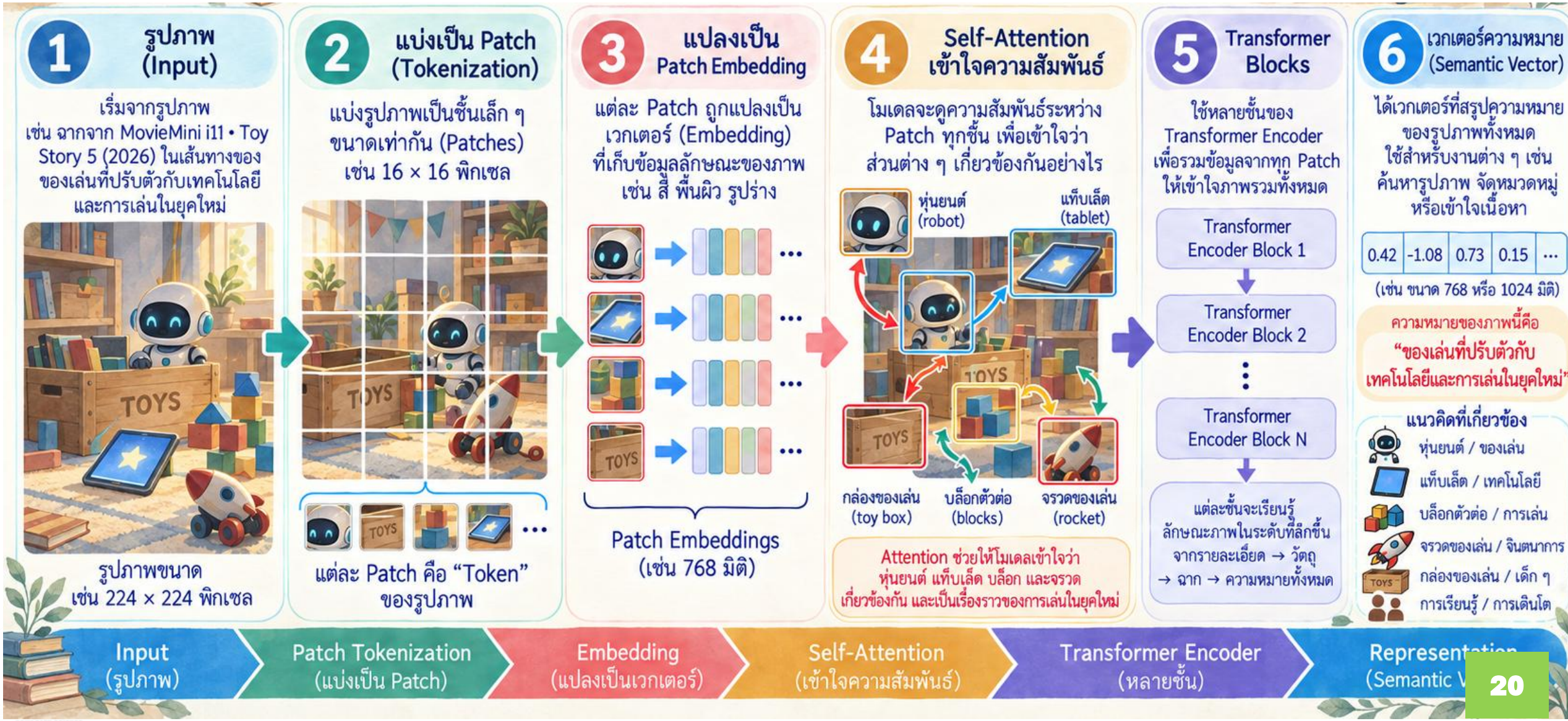


Image Transformation Encoder : Image Patching

1 รูปภาพนำเข้า (Input Image)

รูปภาพขนาด $H \times W \times C$
$= 224 \times 224 \times 3$
(ตัวเลขจำลองเพื่อสาธิต)



$H = 224,\ W = 224,\ C = 3$
(ตัวเลขจำลองเพื่อสาธิต)

2 กำหนดขนาด Patch (Patch Size)

แบ่งรูปภาพเป็นชิ้นเล็ก ๆ ขนาด
$P \times P$ พิกเซล เช่น 16×16



$P = 16$
(ตัวเลขจำลองเพื่อสาธิต)

3 จำนวนแถวและคอลัมน์ของ Patch (Grid Size)

จำนวนแถว $= 224 \div 16 = 14$
จำนวนคอลัมน์ $= 224 \div 16 = 14$



ขนาดกริด (Grid) $= 14 \times 14$
(ตัวเลขจำลองเพื่อสาธิต)

4 จำนวน Patch ทั้งหมด (Number of Patches)

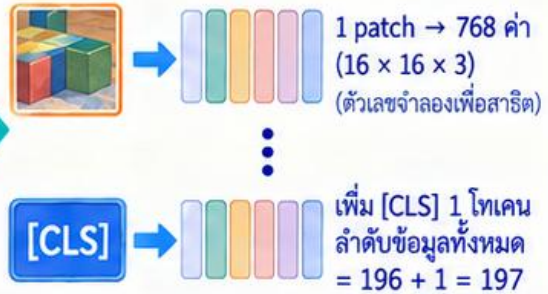
$N = (H/P) \times (W/P)$
$= 14 \times 14 = 196$



$N = 196$ patches
(ตัวเลขจำลองเพื่อสาธิต)

5 แปลงแต่ละ Patch เป็นเวกเตอร์และเพิ่ม [CLS] (Patch Embedding)

แต่ละ Patch ถูกแปลงเป็นเวกเตอร์
ที่มีค่า $P^2 \times C = 16 \times 16 \times 3 = 768$ ค่า
จากนั้นเพิ่มโทเคนพิเศษ [CLS]
ทำให้ความยาวลำดับข้อมูล = 197



ความยาวลำดับข้อมูล (Sequence Length)
$= 197$ tokens
(ตัวเลขจำลองเพื่อสาธิต)

ตัวอย่างขนาดเล็ก (Toy Example)



รูปภาพขนาด 8×8, Patch ขนาด 4×4

- จำนวนแถว $= 8 \div 4 = 2$
- จำนวนคอลัมน์ $= 8 \div 4 = 2$
- จำนวน Patch $= 2 \times 2 = 4$

ลำดับของ Patch (แบบแถวก่อนคอลัมน์)
$1 \rightarrow 2 \rightarrow 3 \rightarrow 4$
(เรียงจากซ้ายไปขวา บนลงล่าง)

สรุปภาพรวม (Overall Flow)



Image
$224 \times 224 \times 3$

196 patch tokens
(แต่ละอันมีค่า 768 ค่า)

197 tokens
(1 [CLS] + 196 patches)

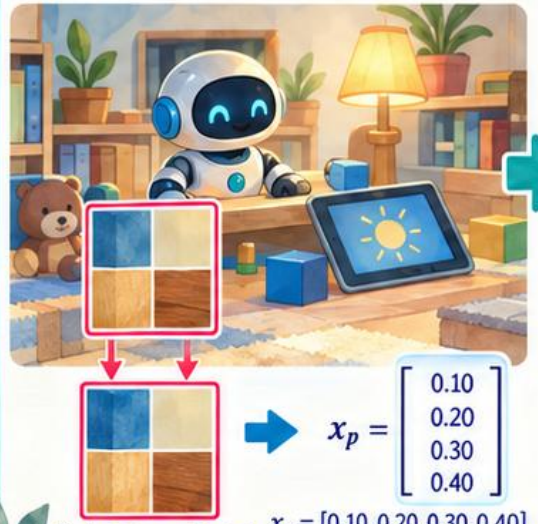
ขั้นตอนเล็ก ๆ ของข้อมูล นำไปสู่พลังของ AI ในอนาคต



Image Transformation Encoder : Image Patch Embedding

1 เลือก Patch และแปลงเป็นเวกเตอร์ (Flatten)

เลือกภาพขนาดเล็ก (Patch)
เช่น 2 × 2 พิกเซล มี 1 ช่องสี
(เพื่อสาธิต) แล้วเรียงเป็นเวกเตอร์



ตัวอย่าง Patch ขนาด 2 × 2 × 1

$$x_p = \begin{bmatrix} 0.10 \\ 0.20 \\ 0.30 \\ 0.40 \end{bmatrix}$$

$x_p = [0.10, 0.20, 0.30, 0.40]$
(ตัวเลขจำลองเพื่อสาธิต)

2 เมทริกซ์ฉายภาพ (Projection Matrix)

ใช้เมทริกซ์น้ำหนัก W_E
และเวกเตอร์ไบอัส b
เพื่อแปลงเวกเตอร์พิกเซล
ไปยังเวกเตอร์มิติต่ำกว่า (D)

$$W_E\,(3 \times 4) = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & -1 & 0 & 0 \end{bmatrix}$$

$$b = \begin{bmatrix} 0.05 \\ -0.05 \\ 0.00 \end{bmatrix}$$

3 คำนวณเวกเตอร์จากการฉายภาพ

คำนวณ $q_p = W_E x_p + b$
ทีละแถว (row)

$$q_1 = 0.10 + 0.30 + 0.05 = \mathbf{0.45}$$

$$q_2 = 0.20 + 0.40 - 0.05 = \mathbf{0.55}$$

$$q_3 = 0.10 - 0.20 = \mathbf{-0.10}$$

$$q_p = \begin{bmatrix} 0.45 \\ 0.55 \\ -0.10 \end{bmatrix}$$

(ตัวเลขจำลองเพื่อสาธิต)

4 เพิ่มเวกเตอร์ตำแหน่ง (Positional Embedding)

สร้างเวกเตอร์ตำแหน่ง p_p
ที่บอกตำแหน่งของ Patch
ในภาพ แล้วนำมาบวก

$$p_p = \begin{bmatrix} 0.01 \\ 0.02 \\ 0.03 \end{bmatrix}$$

(ตัวเลขจำลองเพื่อสาธิต)



แม้ Patch อาจมีลักษณะคล้ายกัน แต่ตำแหน่งในภาพช่วยให้โมเดลรู้ว่าอยู่ตรงไหน!

5 ได้เวกเตอร์ของ Patch (รวมข้อมูลตำแหน่งแล้ว)

นำเวกเตอร์จากการฉายภาพ
มาบวกกับเวกเตอร์ตำแหน่ง

$$e_p = q_p + p_p$$

$$= \begin{bmatrix} 0.45 \\ 0.55 \\ -0.10 \end{bmatrix} + \begin{bmatrix} 0.01 \\ 0.02 \\ 0.03 \end{bmatrix}$$

$$= \begin{bmatrix} 0.46 \\ 0.57 \\ -0.07 \end{bmatrix}$$

(ตัวเลขจำลองเพื่อสาธิต)

เวกเตอร์นี้คือ
"Image Patch Embedding"
ที่โมเดลใช้เป็นข้อมูลนำเข้า!

ในระดับขนาดจริง

หากใช้ Patch ขนาด 16 × 16 พิกเซล
และมี 3 ช่องสี (RGB)
เวกเตอร์ที่แปลงแล้วจะมี 16 × 16 × 3 = 768 ค่า
จากนั้นฉายภาพไปยังมิติ D เช่น 768

สมการโดยรวมของ Image Patch Embedding

$$E = X_{patch} W_E^T + b + E_{position}$$

โดย X_{patch} คือเมทริกซ์ของเวกเตอร์ Patch ทั้งหมด (แต่ละแถวเป็น Patch หนึ่งอัน)

ตัวอย่างที่แสดงเป็นกรณี 4 → 3 มิติ
เพื่อให้ง่ายต่อการเข้าใจ
ตัวเลขทั้งหมดเป็นตัวเลขจำลอง
เพื่อสาธิตแนวคิดเท่านั้น





Small Patches, Big Ideas • เข้าใจหลักการ สร้างอนาคตที่ดีกว่า

Image Transformation Encoder : Image Self-Attention

1 ภาพและ Patch Tokens (Input)

แบ่งภาพเป็น Patch ขนาดเท่ากัน และแทนแต่ละ Patch ด้วยเวกเตอร์ (Token)



แต่ละ Patch แทนด้วยเวกเตอร์ 2 มิติ (ตัวเลขจำลองเพื่อสาธิต)

P1 (หุ่นยนต์) = [1.00, 0.00]
P2 (แท็บเล็ต) = [0.00, 1.00]
P3 (บล็อกของเล่น) = [1.00, 1.00]

2 สร้าง Q, K, V (Linear Projection)

ในการตัวอย่างนี้กำหนด
$W_Q = W_K = W_V = I$
(เมทริกซ์เอกลักษณ์) ดังนั้น
Q = K = V = Patch embedding เดิม

Patch	Q	K	V
P1	$\begin{bmatrix}1.00\\0.00\end{bmatrix}$	$\begin{bmatrix}1.00\\0.00\end{bmatrix}$	$\begin{bmatrix}1.00\\0.00\end{bmatrix}$
P2	$\begin{bmatrix}0.00\\1.00\end{bmatrix}$	$\begin{bmatrix}0.00\\1.00\end{bmatrix}$	$\begin{bmatrix}0.00\\1.00\end{bmatrix}$
P3	$\begin{bmatrix}1.00\\1.00\end{bmatrix}$	$\begin{bmatrix}1.00\\1.00\end{bmatrix}$	$\begin{bmatrix}1.00\\1.00\end{bmatrix}$

เราสนใจว่า P3 มอง Patch อื่นอย่างไร ดังนั้นใช้เวกเตอร์ของ P3 เป็น Query (q)

q = P3 = [1.00, 1.00]
$d_k = 2 \Rightarrow \sqrt{d_k} = \sqrt{2}$

3 คำนวณคะแนนความสำคัญ (Attention Scores)

คำนวณสเกลาร์ดอทโปรดักต์ ระหว่าง query ของ P3 กับ key ของทุก Patch แล้วหารด้วย $\sqrt{d_k} = \sqrt{2}$

j	k_j	$q \cdot k_j$	score = $\frac{q \cdot k_j}{\sqrt{2}}$
P1	$\begin{bmatrix}1.00\\0.00\end{bmatrix}$	1.00	0.7071
P2	$\begin{bmatrix}0.00\\1.00\end{bmatrix}$	1.00	0.7071
P3	$\begin{bmatrix}1.00\\1.00\end{bmatrix}$	2.00	1.4142

scores = [0.7071, 0.7071, 1.4142]
(ตัวเลขจำลองเพื่อสาธิต)

4 แปลงเป็นน้ำหนัก (Softmax)

ใช้ฟังก์ชัน softmax เพื่อแปลงคะแนนเป็นน้ำหนักความสำคัญ (attention weights)

j	score	α_j
P1	0.7071	0.2483
P2	0.7071	0.2483
P3	1.4142	0.5035

α = [0.2483, 0.2483, 0.5035]
ผลรวม (เพราะการปัดเศษ) = 1.0001
ผลรวมที่แท้จริง (ประมาณ) = 1.0000

5 รวมค่าเวกเตอร์ (Weighted Sum)

น้ำหนักคูณกับเวกเตอร์ค่า (V) แล้วรวมกันเป็นเวกเตอร์บริบท (context) ของ P3

$$h_{P3} = \sum_{j=1}^{3} \alpha_j V_j$$

= 0.2483 [1.00, 0.00]
+ 0.2483 [0.00, 1.00]
+ 0.5035 [1.00, 1.00]
= [0.7517, 0.7517]

h_{P3} = [0.7517, 0.7517]
(ตัวเลขจำลองเพื่อสาธิต)

6 ภาพรวมการมองเห็นของ P3 (Visualizing Attention)

ความหนาของลูกศรแสดงขนาดน้ำหนักความสำคัญที่ P3 ให้กับแต่ละ Patch



7 สรุปความหมาย (Interpretation)

Patch ของบล็อกของเล่น (P3) ผสมผสานข้อมูลจากหุ่นยนต์ แท็บเล็ต และตัวมันเอง เพื่อเข้าใจว่าสิ่งของเหล่านี้อยู่ในฉากเดียวกัน และสร้างความหมายของภาพได้ดีขึ้น

8 ข้อควรรู้เพิ่มเติม

- ตัวอย่างนี้ใช้เวกเตอร์ 2 มิติ และเมทริกซ์เอกลักษณ์ เพื่อให้เข้าใจง่าย (ตัวเลขจำลองเพื่อสาธิต)
- โมเดลจริง เช่น Vision Transformer (ViT) ใช้หลายหัวความสนใจ (multi-heads)
- ใช้เมทริกซ์ที่เรียนรู้ได้ (W_Q, W_K, W_V) และเวกเตอร์มิติสูงมาก (เช่น 768 มิติ)
- หลักการเดียวกัน: แต่ละ Patch มอง Patch อื่น เพื่อเข้าใจภาพทั้งภาพ





จากส่วนเล็ก ๆ สู่ความเข้าใจภาพทั้งภาพ — นี่คือพลังของ Self-Attention

Image Transformation Encoder : Transformer Encoder Block

1 ภาพอินพุต (Input Image)



แบ่งภาพเป็นแพทช์ขนาดเท่ากัน
เช่น 16 × 16 พิกเซล (ตัวเลขจำลองเพื่อสาธิต)



ทุกแพทช์ถูกประมวลผลพร้อมกัน
(Parallel Processing)
เป็นลำดับ (Sequence) ของแพทช์

2 Self-Attention (เข้าใจความสัมพันธ์ของภาพทั้งภาพ)

แพทช์ของแท็บเล็ตจะคำนวณความสัมพันธ์กับทุกแพทช์
และได้เวกเตอร์ใหม่ที่มีข้อมูลจากทั้งภาพ

เวกเตอร์อินพุต x
(ของแพทช์แท็บเล็ต)

$$x = [\ 0.6,\ 0.2,\ -0.4\ ]$$

ตัวเลขจำลองเพื่อสาธิต
(เวกเตอร์ 3 มิติ)

→

ผลจาก Self-Attention

$$\text{Attention}(x) = [\ 0.4,\ 0.8,\ -0.6\ ]$$

ตัวเลขจำลองเพื่อสาธิต
(เวกเตอร์ 3 มิติ)

3 Add & Norm (ครั้งที่ 1)

$$z = \text{LayerNorm}(x + \text{Attention}(x))$$

Residual r_1
(บวกแบบข้ามชั้น)

$$r_1 = x + \text{Attention}(x) = [\ 1.0,\ 1.0,\ -1.0\ ]$$

→ **LayerNorm**

$$z = \text{LayerNorm}(r_1) = [0.7071,\ 0.7071,\ -1.4142\ ]$$

ตัวเลขจำลองเพื่อสาธิต

เส้นทางลัด (Residual Connection)

4 FFN (Feed-Forward Network)

แปลงเวกเตอร์ด้วยโครงข่ายประสาทเทียม

ผลจาก FFN

$$\text{FFN}(z) = [0.2929,\ -0.7071,\ 0.4142\ ]$$

ตัวเลขจำลองเพื่อสาธิต

5 Add & Norm (ครั้งที่ 2)

$$h = \text{LayerNorm}(z + \text{FFN}(z))$$

Residual r_2
(บวกแบบข้ามชั้น)

$$r_2 = z + \text{FFN}(z) = [1.0000,\ 0.0000,\ -1.0000]$$

→ **ผลลัพธ์สุดท้าย**

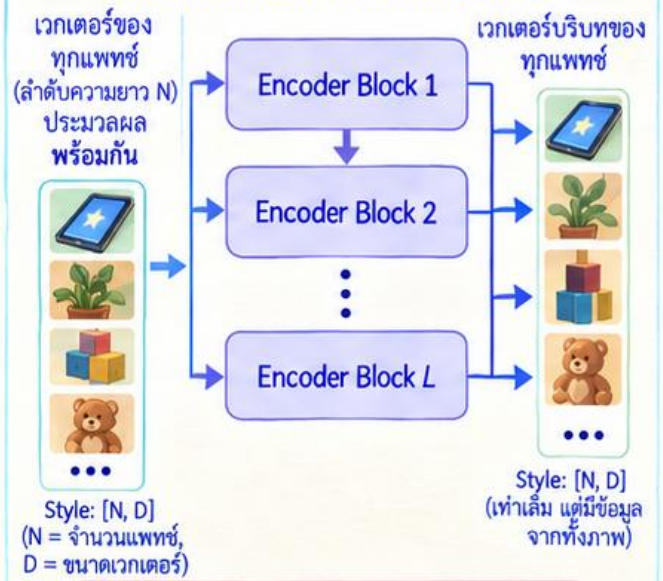
$$h = \text{LayerNorm}(r_2) = [\ 1.2247,\ 0.0000,\ -1.2247\ ]$$

ตัวเลขจำลองเพื่อสาธิต

เส้นทางลัด (Residual Connection)

6 การซ้อนหลายชั้น (Stack of L Blocks)

นำ Encoder Block มาซ้อนกัน L ชั้น
เพื่อให้โมเดลเข้าใจภาพในหลายระดับ



ทุกแพทช์ถูกประมวลผลด้วย Encoder เดียวกัน
และทำงานแบบขนาน (in parallel)

7 ความหมายของเวกเตอร์แท็บเล็ต



เวกเตอร์สุดท้าย h ของแพทช์แท็บเล็ต
คือ "Contextual Patch Vector"
ที่ได้รับข้อมูลจากแพทช์ทั้งหมดในภาพ
เช่น รู้ว่าแท็บเล็ตวางอยู่บนพรม ใกล้ของเล่น
จึงเข้าใจบริบทและความหมายของภาพได้ดี

Image Transformation Encoder : Pooling

1 เวกเตอร์ของแต่ละ Patch (Patch Vectors)

แบ่งภาพเป็นส่วนย่อย (Patch) แล้วแต่ละ Patch จะถูกแปลงเป็นเวกเตอร์คุณลักษณะ (เช่น 4 มิติ)



h_1 : หุ่นยนต์ $[0.1, 0.4, 0.2, 0.3]$



h_2 : แท็บเล็ต $[0.5, 0.2, 0.1, 0.0]$



h_3 : บล็อกตัวต่อ $[0.3, 0.8, 0.6, 0.1]$



h_4 : โคมไฟ $[0.0, 0.1, 0.3, 0.7]$

ตัวเลขทั้งหมดเป็นตัวเลขจำลองเพื่อสาธิต (เวกเตอร์ขนาด 4 มิติ)

แนวคิดสำคัญ

"Pooling รวมข้อมูลจากทุก Patch ให้เป็นเวกเตอร์เดียวที่แทนภาพทั้งภาพ"

แม้จะใช้วิธีต่างกัน แต่เป้าหมายคือได้ Image Vector ที่สื่อความหมายของภาพทั้งหมด

2 Mean Pooling (ค่าเฉลี่ย)

หาค่าเฉลี่ยของเวกเตอร์ทุก Patch

$$z_{\text{mean}} = \frac{1}{N}\sum_{i=1}^{N} h_i$$

ผลลัพธ์ (ตัวเลขจำลองเพื่อสาธิต)

$z_{\text{mean}} = [0.225, 0.375, 0.300, 0.275]$

จุดเด่น: เสถียร ลดสัญญาณรบกวน เหมาะกับภาพที่มีองค์ประกอบหลากหลาย

3 Max Pooling (ค่าสูงสุด)

เลือกค่าที่มากที่สุดในแต่ละมิติ

$$z_{\max}[j] = \max_{i=1,\ldots,N} h_i[j]$$

ผลลัพธ์ (ตัวเลขจำลองเพื่อสาธิต)

$z_{\max} = [0.5, 0.8, 0.6, 0.7]$

จุดเด่น: คงข้อมูลที่เด่นที่สุดในแต่ละมิติ เหมาะกับการเก็บสัญญาณสำคัญ

4 [CLS] Pooling (โทเคนสรุปภาพ)

[CLS] เป็นเวกเตอร์พิเศษที่โมเดลเรียนรู้เพื่อสรุปข้อมูลของทั้งภาพ

$$z_{\text{CLS}} = h_{\text{CLS}}$$

ผลลัพธ์ (ตัวเลขจำลองเพื่อสาธิต)

$h_{CLS} = [0.6, 0.1, 0.1, 0.2]$

จุดเด่น: เป็นสรุปภาพที่เรียนรู้ได้จากข้อมูล เหมาะกับการฝึกแบบธรรมดาที่ซับซ้อน

5 Attention Pooling (ถ่วงน้ำหนักตามความสำคัญ)

คำนวณน้ำหนักความสำคัญของแต่ละ Patch แล้วหาผลรวมถ่วงน้ำหนัก

$$z_{\text{att}} = \sum_{i=1}^{N} \alpha_i h_i \quad \left(\sum_{i=1}^{N} \alpha_i = 1\right)$$

น้ำหนักของแต่ละ Patch (ตัวเลขจำลองเพื่อสาธิต)

$\alpha_1 = 0.2$	$\alpha_2 = 0.1$	$\alpha_3 = 0.5$	$\alpha_4 = 0.2$	$\sum \alpha_i = 1.0$

ผลลัพธ์ (ตัวเลขจำลองเพื่อสาธิต)

$z_{\text{att}} = [0.22, 0.52, 0.41, 0.25]$

จุดเด่น: เน้นส่วนที่เกี่ยวข้องกับงาน ให้น้ำหนักตามบริบท เหมาะกับภาพที่มีบางส่วนสำคัญ

สรุปข้อเปรียบเทียบ

Mean	✓ เสถียร ✓ ลดสัญญาณรบกวน ✓ เหมาะกับภาพทั่วไป
Max	✓ เก็บค่าที่เด่นที่สุด ✓ เหมาะกับคุณลักษณะที่ต้องการความชัดเจน
[CLS]	✓ เรียนรู้ได้จากข้อมูล ✓ สรุปภาพแบบรวมถึง ✓ เหมาะกับงานที่ซับซ้อน
Attention	✓ เน้นส่วนที่เกี่ยวข้อง ✓ ยืดหยุ่นตามบริบท ✓ มักให้ประสิทธิภาพสูง

ไม่มีวิธีใดดีที่สุดเสมอไป ควรเลือกตามลักษณะของงานและข้อมูล

Patch vectors H (ขนาด $N \times 4$)



h_1 h_2 h_3 h_4 ...

→ **Pooling** (Mean / Max / [CLS] / Attention)

→ **Image vector z** (ขนาด 4 มิติ) $[0.22, 0.52, 0.41, 0.25]$ (ตัวเลขจำลองเพื่อสาธิต)

→ **Projection** $(z' = Wz)$ แปลงสู่เวกเตอร์ในมิติที่เหมาะสมสำหรับงานปลายน้ำ

→ **Cosine Similarity** $sim(z', t') = \frac{z' \cdot t'}{\|z'\|\|t'\|}$ ใช้เปรียบเทียบความคล้ายลึกลับกับข้อความหรือภาพอื่น

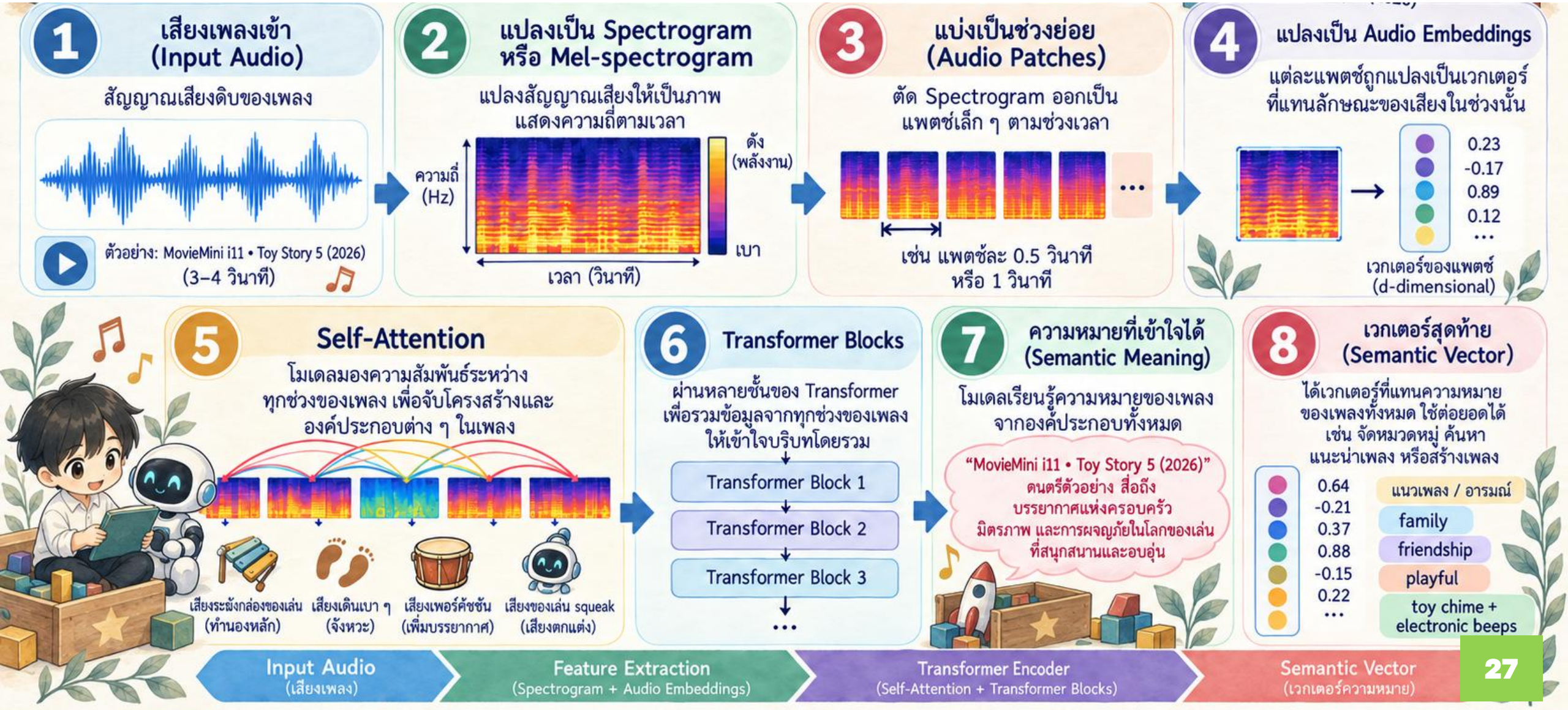


AI ช่วยให้เราเห็นภาพ... และเข้าใจภาพได้ลึกยิ่งขึ้น

หมายเหตุ: ตัวเลขทั้งหมดเป็นตัวเลขจำลองเพื่อสาธิต (ตัวอย่างเวกเตอร์ขนาด 4 มิติ)

Audio Transformation Encoder

ภาพรวม Audio Transformation Encoder



Audio Transformation Encoder : Patching

แบ่งเสียงตามแกนความถี่ x เวลา

1 คลิปเสียงและการสุ่มตัวอย่าง

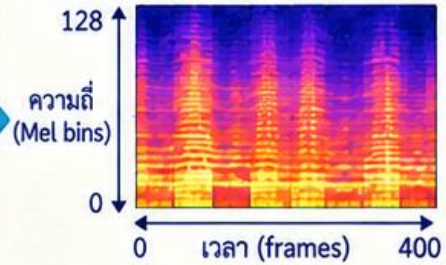
ความยาว 4 วินาที
อัตราการสุ่มตัวอย่าง 16,000 Hz



จำนวนตัวอย่าง (samples)
4 × 16,000 = 64,000 samples
(ตัวเลขจำลองเพื่อสาธิต)

2 แปลงเป็น Mel-spectrogram

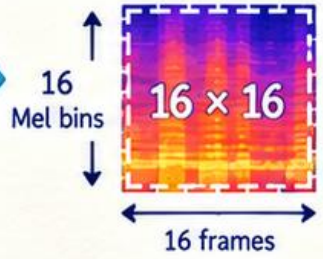
ใช้ STFT และ Mel filterbank
ได้ Mel-spectrogram



ขนาด 128 Mel bins × 400 frames
(ตัวเลขจำลองเพื่อสาธิต)

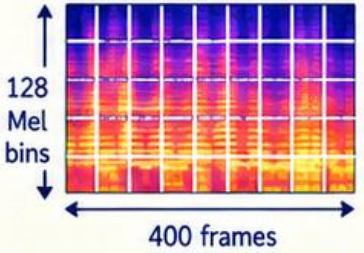
3 ขนาดของ Patch

กำหนดขนาด patch
เป็น 16 × 16
(16 Mel bins × 16 frames)



4 จำนวน Patch ในแต่ละแกน

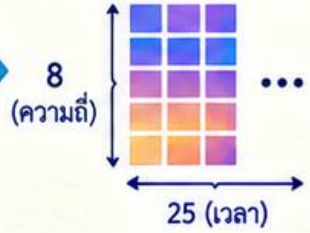
ความถี่: 128 / 16 = 8
เวลา: 400 / 16 = 25



ได้ 8 × 25 ช่อง
ของ patch

5 จำนวน Audio Patches

จำนวน patch ทั้งหมด
N = 8 × 25 = 200
(ตัวเลขจำลองเพื่อสาธิต)



รวม 200 patches

6 เวกเตอร์ของแต่ละ Patch

แต่ละ patch มีค่า 16 × 16
= 256 ค่า (flattened)



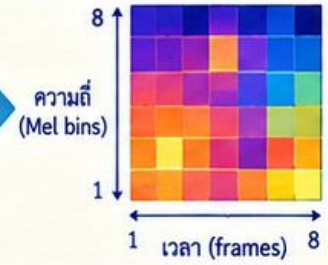
เพิ่ม [CLS] token 1 ตัว
ได้ทั้งหมด 201 tokens
(ตัวเลขจำลองเพื่อสาธิต)

ตัวอย่างง่าย ๆ : จากสเปกโตรแกรม 8×8 สู่ 4 patches

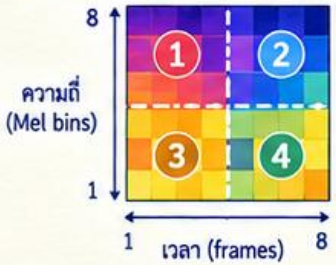
1. คลิปเสียง (ตัวอย่าง)



2. Mel-spectrogram ขนาด 8 × 8



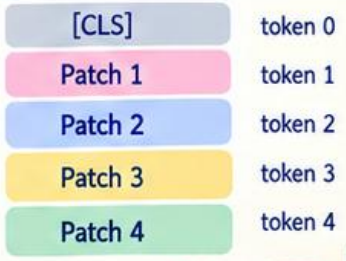
3. แบ่งเป็น patch 4 × 4



4. ได้ 4 patches (เรียงตามเวลา-ความถี่)



5. แปลงเป็นเวกเตอร์ (flatten) + [CLS]



แต่ละ patch จับลวดลายของเสียงในช่วงความถี่และเวลาสั้น ๆ เช่น เสียงระฆังช่วงต้น เสียงกลองช่วงกลาง หรือเสียงเดินช่วงปลาย
เมื่อนำ patch ทั้งหมดมาเรียงกัน โมเดลจะมองเห็นโครงสร้างของเสียงทั้งคลิป

เสียงเล็ก ๆ ก็เล่าเรื่องใหญ่

Audio Transformation Encoder : Patch Embedding

Linear Project + Time-Frequency Position

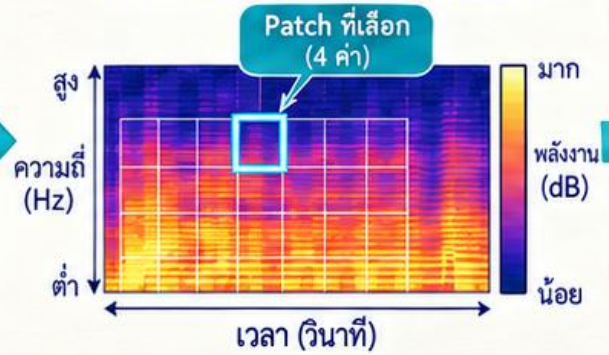
1 เสียงในชีวิตประจำวัน (Time-Domain Audio)

สัญญาณเสียงในรูปคลื่นตามเวลา (waveform)



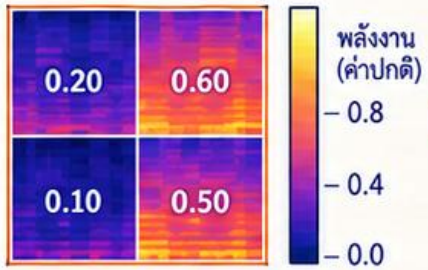
2 แปลงเป็น Spectrogram (Time-Frequency Representation)

แปลงสัญญาณเสียงเป็นภาพในโดเมนเวลา–ความถี่ แสดงพลังงานของเสียง



3 ตัด Patch จาก Spectrogram

แบ่งภาพเป็นแพตช์ขนาดเล็ก เช่น 2×2 (ในตัวอย่างนี้)



Spectrogram Patch ขนาด 2×2 (ตัวเลขจำลองเพื่อสาธิต)

4 แปลงเป็นเวกเตอร์ (Linear Projection)

ทำให้แพตช์เป็นเวกเตอร์แถว แล้วฉายเชิงเส้นด้วยน้ำหนัก W_E และบวกไบอัส

4.1 Flatten แพตช์

$$x_p = [0.20, 0.60, 0.10, 0.50] \quad \text{(ขนาด 4)}$$

4.2 เมทริกซ์น้ำหนักและไบอัส (ตัวเลขจำลองเพื่อสาธิต)

$$W_E\,(3\times4) = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & -1 & 0 & 0 \end{bmatrix} \qquad b = \begin{bmatrix} 0.05 \\ -0.05 \\ 0.00 \end{bmatrix}$$

5 คำนวณเวกเตอร์แพตช์ (ตัวอย่าง 4 → 3 มิติ)

คำนวณ $q = W_E x + b$ ทีละองค์ประกอบ

$$q_1 = 0.20 + 0.10 + 0.05 = 0.35$$
$$q_2 = 0.60 + 0.50 - 0.05 = 1.05$$
$$q_3 = 0.20 - 0.60 = -0.40$$

ดังนั้น $q = [0.35, 1.05, -0.40]$ (ขนาด 3)

6 เพิ่มตำแหน่งเวลา–ความถี่ (Time-Frequency Position)

สร้างเวกเตอร์ตำแหน่ง p ที่บอกตำแหน่งของแพตช์ในแกนเวลาและความถี่ (ตัวเลขจำลองเพื่อสาธิต)

$$p = [0.01, 0.02, 0.03] \quad \text{(ขนาด 3)}$$

ตำแหน่งช่วยให้แบบจำลองรู้ว่า
- ความถี่สูงหรือต่ำ (แกนความถี่)
- เกิดขึ้นในช่วงต้นหรือปลาย (แกนเวลา)

7 ได้เวกเตอร์แพตช์สุดท้าย

บวกเวกเตอร์ตำแหน่งกับผลฉายเชิงเส้น

$$e = q + p = [0.35, 1.05, -0.40] + [0.01, 0.02, 0.03]$$

$$e = [0.36, 1.07, -0.37] \quad \text{(ขนาด 3)}$$

นี่คือ Audio Patch Embedding ของแพตช์นี้ (ตัวเลขจำลองเพื่อสาธิต)

8 ภาพรวมสูตรและการใช้งานจริง

สูตรโดยรวม

$$E_{\text{audio}} = X_{\text{patch}} W_E^T + b + E_{\text{time-frequency}}$$

ในงานจริง
- Spectrogram patch ขนาด 16 × 16 มีค่า 256 ค่า (16 × 16 = 256)
- ฉายเป็นเวกเตอร์ขนาด D มิติ ด้วย Linear Projection
- แล้วบวกเวกเตอร์ตำแหน่งเวลา–ความถี่ เพื่อให้ได้ Audio Patch Embedding
- ตัวอย่างในภาพใช้มิติ 4 → 3 เพื่อให้คำนวณได้ง่าย (ตัวเลขจำลองเพื่อสาธิต)



เสียง (Waveform) คลื่นตามเวลา → Spectrogram ภาพเวลา–ความถี่ → Patch แพตช์ขนาดเล็ก → Linear Projection ฉายเป็นเวกเตอร์ → + Time-Frequency Position ใส่ตำแหน่งเวลา–ความถี่ → Audio Patch Embedding เวกเตอร์ที่แบบจำลองเข้าใจได้

เสียงเล็ก ๆ ก็มีความหมายใหญ่ได้

“ จากเสียงในโลกจริง สู่ความเข้าใจของ AI ”

Audio Transformation Encoder : Self-Attention

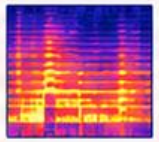
เชื่อมรูปแบบเสียงข้ามเวลาในเพลหรือคลิปเสียง

1 ช่วงเสียงและตัวแทนเวกเตอร์

แบ่งคลิปเสียงเป็นช่วงสั้น ๆ (Audio Patches)
และแทนแต่ละช่วงด้วยเวกเตอร์คุณลักษณะ

A1 เสียงระฆังของเล่น (Toy Chime)



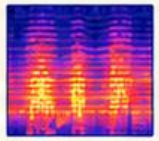


เวกเตอร์คุณลักษณะ (ตัวเลขจำลองเพื่อสาธิต)

$$A1 = \begin{bmatrix} 1.00 \\ 0.50 \end{bmatrix}$$

A2 เสียงเดินเท้า (Footsteps)

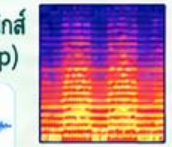




$$A2 = \begin{bmatrix} 0.00 \\ 1.00 \end{bmatrix}$$

A3 เสียงบี๊บอิเล็กทรอนิกส์ (Electronic Beep)





$$A3 = \begin{bmatrix} 1.00 \\ 1.00 \end{bmatrix}$$

ภาพ Spectrogram เป็นภาพประกอบเพื่อสื่อแนวคิด
ลักษณะเสียงจริงอาจแตกต่างออกไป

2 สร้าง Q, K, V จากเวกเตอร์คุณลักษณะ

ใช้เมทริกซ์การฉาย (Projection) เพื่อได้ Query (Q), Key (K) และ Value (V)
ในกาตัวอย่างนี้กำหนด $W_Q = W_K = W_V = I$ และ $d_k = 2$

ช่วงเสียง	เวกเตอร์ x	Query (Q)	Key (K)	Value (V)
A1 (Chime)	[1.00, 0.50]	[1.00, 0.50]	[1.00, 0.50]	[1.00, 0.50]
A2 (Footsteps)	[0.00, 1.00]	[0.00, 1.00]	[0.00, 1.00]	[0.00, 1.00]
A3 (Beep)	[1.00, 1.00]	[1.00, 1.00]	[1.00, 1.00]	[1.00, 1.00]

$W_Q = W_K = W_V = I$ (เมทริกซ์เอกลักษณ์)
$d_k = 2$
ตัวเลขทั้งหมดเป็นค่าจำลองเพื่อสาธิตแนวคิด

3 คำนวณความคล้ายด้วย Dot Product (แบบปรับสเกล)

ให้ช่วง A1 (Toy Chime) เป็น Query: $q = [1.00, 0.50]$
คำนวณคะแนนความเหมือนกับทุกช่วง: $s_j = (q \cdot k_j)/\sqrt{d_k}$ โดย $\sqrt{d_k} = \sqrt{2}$

A1 A1 vs A1 (same patch)
$(1.00\times1.00 + 0.50\times0.50)/\sqrt{2}$
$= 0.8839$

A2 A1 vs A2 (footsteps)
$(1.00\times0.00 + 0.50\times1.00)/\sqrt{2}$
$= 0.3536$

A3 A1 vs A3 (electronic beep)
$(1.00\times1.00 + 0.50\times1.00)/\sqrt{2}$
$= 1.0607$

scaled scores = [0.8839, 0.3536, 1.0607] (ตัวเลขจำลองเพื่อสาธิต)

4 แปลงเป็นน้ำหนักด้วย Softmax

ใช้ Softmax เพื่อแปลงคะแนนให้เป็นน้ำหนักความสนใจ
ที่รวมกันได้ 1

$$\alpha_j = \frac{\exp(s_j)}{\sum_i \exp(s_j)}$$

น้ำหนักความสนใจจาก A1 ไปยังแต่ละช่วง

A1	A2	A3
α_1	α_2	α_3
0.3595	0.2115	0.4290

softmax α = [0.3595, 0.2115, 0.4290]
(ตัวเลขจำลองเพื่อสาธิต)

5 รวมข้อมูลด้วยน้ำหนัก (Weighted Sum)

คำนวณเวกเตอร์บริบทของช่วง A1 จากค่า Value ของทุกช่วง

$$h_{A1} = \sum_j \alpha_j V_j$$

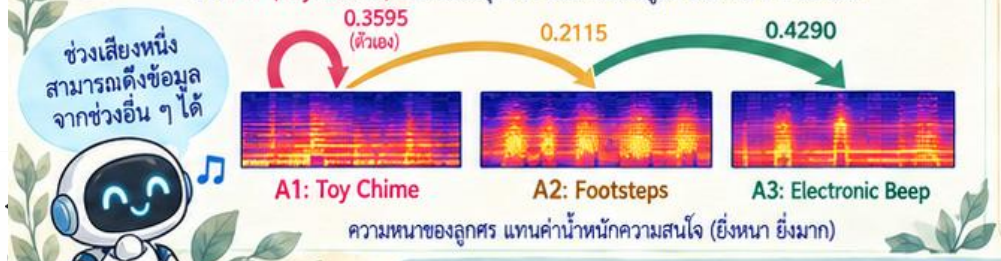
$h_{A1} = 0.3595\,[1, 0.5] + 0.2115\,[0, 1] + 0.4290\,[1, 1]$
$= [\ 0.7885,\ 0.8203\ ]$
(ตัวเลขจำลองเพื่อสาธิต)

เวกเตอร์บริบทของ A1 ที่รวมข้อมูลจากช่วงอื่น

$$h_{A1} = \begin{bmatrix} 0.7885 \\ 0.8203 \end{bmatrix}$$

6 ภาพรวมการเชื่อมโยงข้ามช่วงเสียง

ช่วง A1 (Toy Chime) มองไปยังทุกช่วง และรวมข้อมูลตามน้ำหนักความสนใจ



7 สิ่งที่ได้และความหมาย



ผลลัพธ์: ช่วงเสียงของเล่นระฆัง (Chime) ได้รับข้อมูลเพิ่มเติมจากจังหวะการเดิน (Footsteps) และเสียงบี๊บอิเล็กทรอนิกส์ (Beep) ทำให้เข้าใจบริบทของเสียงได้ดีขึ้น ทั้งลักษณะเสียงและจังหวะ

8 ข้อสังเกตสำคัญ

- Self-Attention ช่วยให้แต่ละช่วงเสียงมองเห็นช่วงอื่น ๆ และเชื่อมรูปแบบข้ามเวลา
- ผลลัพธ์ผสมทั้งลักษณะเสียง (timbre) และจังหวะ (rhythm) เข้าด้วยกัน
- ตัวอย่างนี้ใช้เวกเตอร์ 2 มิติและเมทริกซ์เอกลักษณ์ (ตัวเลขจำลองเพื่อสาธิต)
- โมเดลจริงใช้การฉายเวกเตอร์ที่เรียนรู้ได้ หลายมิติ และหลายหัวความสนใจ (multi-head)



หมายเหตุ: ตัวเลขทั้งหมดเป็นค่าจำลองเพื่อสาธิตแนวคิด
โมเดลจริงใช้การฉายเวกเตอร์ที่เรียนรู้ได้และหลายหัวความสนใจ

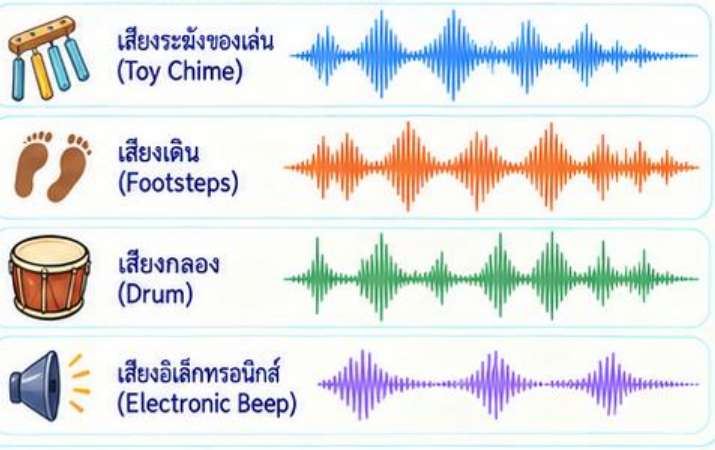
"ฟังให้ไกลกว่าช่วงของตัวเอง แล้วเสียงทั้งหมดจะเชื่อมโยงกันเป็นหนึ่งเดียว"

Audio Transformation Encoder : Transformer Encoder Block

ให้โมเดลเข้าใจเสียงทั้งรายละเอียดและภาพรวม

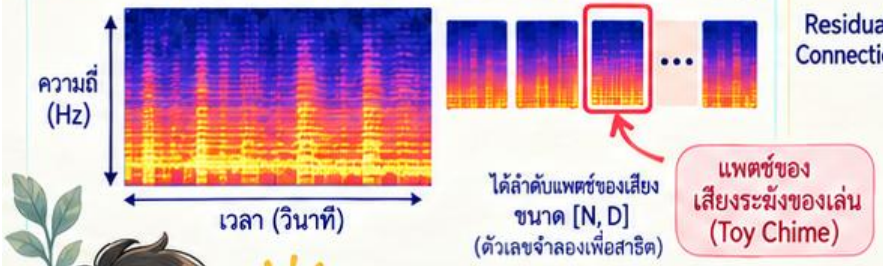
1 ตัวอย่างสัญญาณเสียง (Input Audio)

คลื่นเสียงในชีวิตประจำวัน



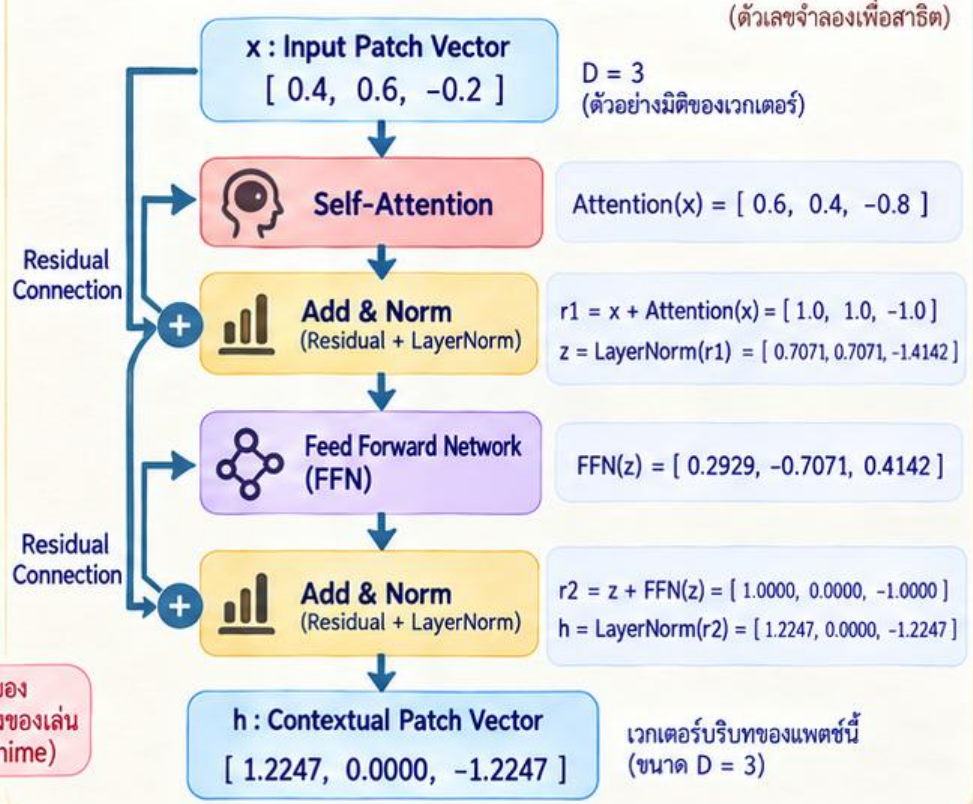
2 แปลงเป็น Spectrogram และแบ่งเป็น Patches

แปลงเสียงเป็นภาพสเปกโตรแกรม แล้วตัดเป็นแพตช์เล็ก ๆ



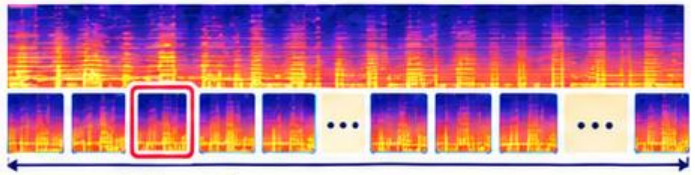
3 ภายใน Transformer Encoder Block

ตัวอย่างการคำนวณสำหรับ 1 แพตช์ (เสียงระฆังของเล่น) (ตัวเลขจำลองเพื่อสาธิต)



4 ลำดับแพตช์และการประมวลผลแบบขนาน

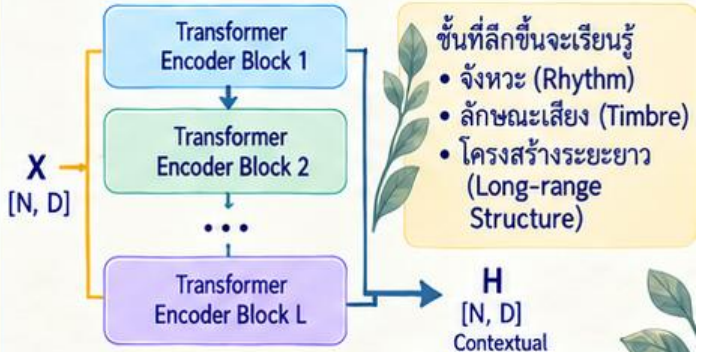
แพตช์ทั้งหมดถูกประมวลผลพร้อมกัน (in parallel)



ลำดับแพตช์ (Token Sequence) ขนาด [N, D]
N = จำนวนแพตช์, D = มิติของเวกเตอร์ (ตัวเลขจำลองเพื่อสาธิต)

5 สแต็กของ Transformer Encoder Blocks

ใช้ L ชั้น เพื่อเรียนรู้ลักษณะเสียงที่ซับซ้อนขึ้น



6 สรุปแนวคิดสำคัญ

- **Self-Attention** จับความสัมพันธ์ระหว่างแพตช์ทั้งหมด
- **Residual + LayerNorm** ช่วยให้การฝึกโมเดลเสถียรและมีประสิทธิภาพ
- **Deep Stacks** ชั้นที่ลึกขึ้นเรียนรู้จังหวะ ลักษณะเสียง และโครงสร้างระยะยาว
- **ผลลัพธ์** ได้เวกเตอร์ที่เข้าใจบริบทของเสียง นำไปใช้ในงานต่าง ๆ เช่น การจำแนกเสียง การค้นหา หรือการสร้างเสียง



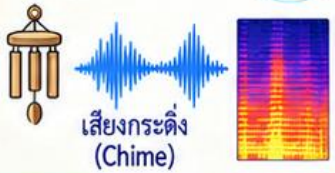


Audio Transformation Encoder : Pooling

สรุปเสียงทั้งคลิปเป็น 1 Vetor

1 Audio Patch Vectors จากช่วงเสียงต่าง ๆ

เราแบ่งเสียงทั้งคลิปออกเป็นช่วงสั้น ๆ (audio patches) แล้วแปลงเป็นเวกเตอร์คุณลักษณะ (ตัวเลขจำลองเพื่อสาธิต)



h_1 = [0.6, 0.1, 0.2, 0.3]



h_2 = [0.2, 0.7, 0.1, 0.0]



h_3 = [0.4, 0.3, 0.8, 0.1]



h_4 = [0.1, 0.2, 0.3, 0.9]

2 วิธี Pooling เพื่อได้ Audio Vector

รวมข้อมูลจากทุกช่วงเสียงให้เป็นเวกเตอร์เดียว z_{audio} (ตัวเลขจำลองเพื่อสาธิต, มิติ 4D)

2.1 Mean Pooling

เฉลี่ยเวกเตอร์จากทุกช่วงเสียง

$$z = \frac{1}{N}\sum_{i=1}^{N} h_i$$

z_{audio} = [0.325, 0.325, 0.350, 0.325]

สรุปข้อมูลโดยรวมของทั้งคลิป
เหมาะกับเสียงที่มีหลายเหตุการณ์

2.2 Max Pooling

เลือกค่ามากที่สุดในแต่ละมิติ

$$z_j = \max_i (h_{i,j})$$

z_{audio} = [0.6, 0.7, 0.8, 0.9]

เน้นคุณลักษณะที่เด่นที่สุด
เหมาะเมื่อต้องการจับสัญญาณเด่น
แม้เกิดเพียงช่วงสั้น ๆ

2.3 [CLS] Token Vector

ใช้เวกเตอร์พิเศษที่เรียนรู้ได้

[CLS] → $z = h_{[CLS]}$

z_{audio} = [0.5, 0.2, 0.4, 0.3]

เวกเตอร์สรุปที่โมเดลเรียนรู้เอง
เหมาะกับงานทั่วไป ให้ผลดีและรวดเร็ว

2.4 Attention Pooling

ถ่วงน้ำหนักตามความสำคัญของแต่ละช่วง

$$z = \sum_{i=1}^{N} \alpha_i h_i \quad \text{และ} \quad \sum_{i=1}^{N} \alpha_i = 1$$

น้ำหนัก (α_i) = [0.4, 0.1, 0.2, 0.3] (ผลรวม = 1.0)

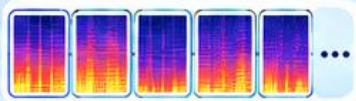
z_{audio} = [0.37, 0.23, 0.34, 0.41]

เน้นช่วงที่สำคัญ เช่น เสียงกระดิ่ง (0.4)
และเสียงบี๊บ (0.3) มีส่วนมากที่สุด

3 ภาพรวมการใช้งาน: จากเสียงทั้งคลิปสู่การประยุกต์ใช้



Audio Patches (จากคลิปเสียง)



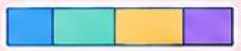
เช่น กระดิ่ง → เดิน → กลอง → บี๊บ

→ **Patch Vectors** h_1, h_2, h_3, h_4 ...

→ **Pooling** (Mean / Max / [CLS] / Attention)



→ **Audio Vector** z_{audio}



เช่น [0.37, 0.23, 0.34, 0.41]
(มิติ 4D)

→ **Projection** (Linear Layer)

$z' = Wz_{audio}$



แปลงเป็นมิติที่เหมาะสม
เช่น 256D

→ **การใช้งานต่อไป**

- Cosine Similarity (เปรียบเทียบความคล้าย)
- หรือ Multimodal Fusion (รวมกับข้อมูลอื่น เช่น ข้อความ, ภาพ)

Audio Pooling = สะพานจากเสียงที่มีหลายช่วงเวลา สู่เวกเตอร์เดียวที่ใช้งานได้หลากหลาย

ตัวเลขทั้งหมดเป็นตัวอย่างจำลองเพื่อการสอน (4 มิติ)

Video Transformation Encoder

ภาพรวม Video Transformation Encoder

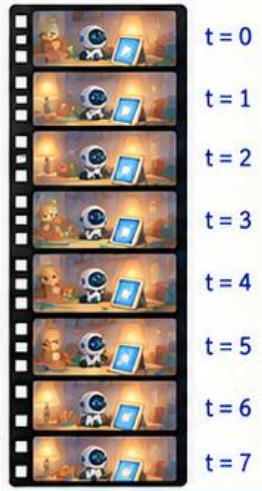


Video Transformation Encoder : Video Patching

แบ่งวิดีโอเป็นท่อของแพตช์ ในมิติทั้งพื้นที่และเวลา

1 วิดีโออินพุต (Input Video)

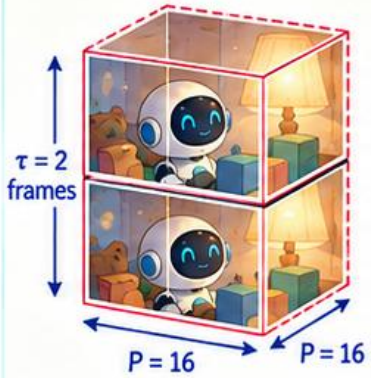
ขนาดวิดีโอ $T \times H \times W \times C$
$= 8 \times 224 \times 224 \times 3$
(ตัวเลขจำลองเพื่อสาธิต)



$T = 8$ เฟรม
$H = 224$, $W = 224$, $C = 3$
(ตัวเลขจำลองเพื่อสาธิต)

2 ขนาดของ Tubelet (Tubelet Size)

$\tau \times P \times P$
$= 2$ frames $\times 16 \times 16$ pixels
(ตัวเลขจำลองเพื่อสาธิต)



หนึ่ง Tubelet ครอบคลุม
2 เฟรมต่อเนื่อง และพื้นที่
16 × 16 พิกเซล ในแต่ละเฟรม

3 จำนวนกลุ่มเวลา (Time Groups)

$\frac{T}{\tau} = \frac{8}{2} = 4$
(ตัวเลขจำลองเพื่อสาธิต)



4 กรีดเชิงพื้นที่ (Spatial Grid)

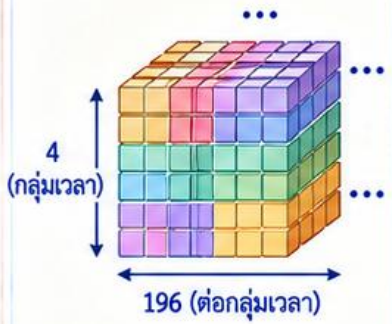
$(H / P) \times (W / P)$
$= (224 / 16) \times (224 / 16)$
$= 14 \times 14 = 196$
(ตัวเลขจำลองเพื่อสาธิต)



แต่ละเฟรมถูกแบ่งเป็น
$14 \times 14 = 196$ แพตช์

5 จำนวน Tubelets ทั้งหมด

$N = 4 \times 196 = 784$
(ตัวเลขจำลองเพื่อสาธิต)



รวมทั้งหมด
784 Tubelets

6 ขนาดของโทเคนและ [CLS] (Flatten & [CLS])

แต่ละ Tubelet มีค่า
$2 \times 16 \times 16 \times 3$
$= 1{,}536$ ค่าดิบ (raw values)
(ตัวเลขจำลองเพื่อสาธิต)



1,536 ค่าดิบ → 1 โทเคน
+
[CLS]
เพิ่มโทเคนพิเศษ [CLS]
สำหรับแทนวิดีโอทั้งคลิป

จำนวนโทเคนทั้งหมด
$= 784 + 1 = 785$ โทเคน
(ตัวเลขจำลองเพื่อสาธิต)

ทำไมต้องใช้ Tubelets ?

✗ การแบ่งแพตช์แบบรายเฟรม (Per-frame Patching)



- มองแต่ข้อมูลในแต่ละเฟรม แยกกันอิสระ
- สูญเสียความเชื่อมโยงของการเคลื่อนไหวตามเวลา

✓ การแบ่งแบบ Tubelets (Spatiotemporal Patching)



- รักษาความเชื่อมโยงของพื้นที่เดียวกันในหลายเฟรม
- ช่วยให้โมเดลเข้าใจการเปลี่ยนแปลงและการเคลื่อนไหวระยะสั้น

ตัวอย่างขนาดเล็ก (Toy Example)

วิดีโอ 4 เฟรม ขนาด 8 × 8 พิกเซล
ใช้ Tubelet $2 \times 4 \times 4$



- จำนวนกลุ่มเวลา $= 4 / 2 = 2$
- กริดเชิงพื้นที่ $= (8 / 4) \times (8 / 4)$
  $= 2 \times 2 = 4$
- จำนวน Tubelets $= 2 \times 4 = 8$

(ตัวเลขจำลองเพื่อสาธิต)

ได้ 8 Tubelets ที่ครอบคลุมทั้งพื้นที่และเวลา

Good Ideas Grow Here ♥

Play Learn Create Grow ♥

Video Transformation Encoder : Patch Embedding

Linear Project + Spatial & Temporal Position

1 เลือก Tubelet และแปลงเป็นเวกเตอร์หนึ่งมิติ (Flatten)

เลือก Tubelet ขนาด 2 เฟรม ขนาดเล็ก 1×1 พิกเซล (มี 3 แชนแนล)



Flatten เป็นเวกเตอร์

$$x = [0.20,\ 0.40,\ 0.60,\ 0.80]$$
t0:R t0:G t1:R t1:G

(ตัวเลขจำลองเพื่อสาธิต)

2 เมทริกซ์ฉายเชิงเส้น (Linear Projection)

ใช้เมทริกซ์น้ำหนัก W_E ขนาด 3 × 4 และเวกเตอร์ไบแอส b ขนาด 3

$$W_E\,(3 \times 4) = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & -1 & 0 & 0 \end{bmatrix}$$

$$b = \begin{bmatrix} 0.05 \\ -0.05 \\ 0.00 \end{bmatrix}$$

แปลงเวกเตอร์ 4 มิติ ให้เป็นเวกเตอร์ 3 มิติ

(ตัวเลขจำลองเพื่อสาธิต)

3 คำนวณเวกเตอร์ฉาย (Linear Projection Result)

$$q = W_E x + b$$

$$q_1 = 0.20 + 0.60 + 0.05 = 0.85$$

$$q_2 = 0.40 + 0.80 - 0.05 = 1.15$$

$$q_3 = 0.20 - 0.40 = -0.20$$

$$q = [0.85,\ 1.15,\ -0.20]$$

(ตัวเลขจำลองเพื่อสาธิต)

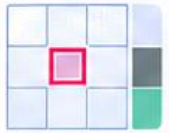
4 เวกเตอร์ตำแหน่งเชิงพื้นที่และเวลา

สร้างเวกเตอร์ตำแหน่งเชิงพื้นที่-เวลา (Spatial-Temporal Position)

$$p = [0.01,\ 0.02,\ 0.03]$$

(ตัวเลขจำลองเพื่อสาธิต)

ตำแหน่งเชิงพื้นที่ (Spatial)



บอกว่า Patch / Tubelet นี้อยู่ "ตำแหน่งไหน" ในภาพ (เช่น ซ้ายบน, ตรงกลาง, ขวาล่าง)

ตำแหน่งเชิงเวลา (Temporal)



บอกว่า Patch / Tubelet นี้ เกิดขึ้น "เมื่อไร" ในลำดับเวลา (เช่น เฟรมที่ 0, 1, 2, ...)

5 รวมกันเป็น Video Tubelet Embedding

$$e = q + p$$

$$e_1 = 0.85 + 0.01 = 0.86$$

$$e_2 = 1.15 + 0.02 = 1.17$$

$$e_3 = -0.20 + 0.03 = -0.17$$

$$e = [0.86,\ 1.17,\ -0.17]$$

(ตัวเลขจำลองเพื่อสาธิต)

สูตรโดยรวม

$$E_{video} = X_{tubelet} W_E^T + b + E_{space} + E_{time}$$

ขนาดจริงในงานปฏิบัติ

Tubelet ขนาด 2 × 16 × 16 × 3
มีค่า 1,536 ค่า (2 เฟรม × 16 สูง × 16 กว้าง × 3 แชนแนล)
ซึ่งจะถูกฉายไปยังเวกเตอร์ขนาด D มิติ

สรุปแนวคิด

แต่ละ Video Tubelet Embedding รวมข้อมูล 3 ส่วน
1) เนื้อหาจากพิกเซล (Linear Projection)
2) ตำแหน่งในภาพ (Spatial Position)
3) ลำดับเวลาในวิดีโอ (Temporal Position)
ทำให้แบบจำลองเข้าใจทั้ง "คืออะไร" "อยู่ที่ไหน" และ "เกิดขึ้นเมื่อไร"

หมายเหตุ

- ตัวอย่างนี้ใช้เวกเตอร์ขนาดเล็ก 4 → 3 มิติ เพื่อสาธิตแนวคิดเท่านั้น
- ตัวเลขทั้งหมดเป็น "ตัวเลขจำลองเพื่อสาธิต" ไม่ได้มาจากข้อมูลจริง
- ในงานจริง D อาจมีขนาดใหญ่ขึ้น (เช่น 768, 1024 เป็นต้น)
- ตำแหน่งเชิงพื้นที่และเวลา มักเรียนรู้ผ่านเวกเตอร์ หรือการเข้ารหัสแบบไซน์/เรียนรู้ได้





Technology today • Brighter learning tomorrow

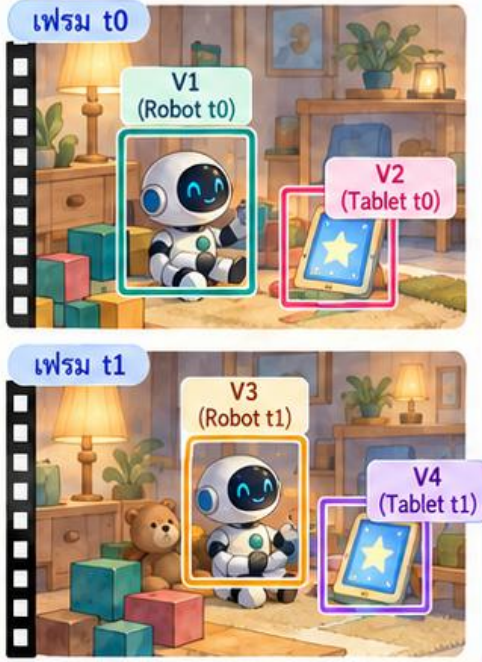
COMPUTER SCIENCE

Video Transformation Encoder : Self-Attention

เชื่อมโยงความสัมพันธ์ พื้นที่ในภาพเดียวกัน (Spatial) และ เวลาข้ามเฟรม

1 วิดีโอและ Tubelet Tokens (Video & Tubelet Tokens)

แบ่งวิดีโอเป็น Tubelet Tokens แต่ละอันแทนแพตช์เล็ก ๆ ในช่วงเวลาและตำแหน่งหนึ่ง ๆ



2 ความสัมพันธ์ที่เชื่อมโยง (Spatial & Temporal Links)

ความสัมพันธ์เชิงพื้นที่ (ในเฟรมเดียวกัน)

ความสัมพันธ์เชิงเวลา (วัตถุเดียวกันข้ามเฟรม)



3 แทนด้วยเวกเตอร์ 2 มิติ (Token Representations)

(ตัวเลขจำลองเพื่อสาธิต)

V1 (Robot t0)	[1.00, 0.00]
V2 (Tablet t0)	[0.00, 1.00]
V3 (Robot t1)	[1.00, 0.50]
V4 (Tablet t1)	[0.50, 1.00]

ตั้งค่าในการคำนวณ

$W_Q = W_K = W_V = I$ (เมทริกซ์เอกลักษณ์)

$d_k = 2$

เลือก Query คือ V3 (Robot t1)

$q = [1.00, 0.50]$

4 คำนวณ Self-Attention

(จาก Q, K, V → score → softmax → weighted sum)

4.1 คำนวณคะแนนความสัมพันธ์ (Scaled Dot-Product)

$$score_i = \frac{q \cdot k_i}{\sqrt{d_k}} \quad (d_k = 2)$$

Token	$q \cdot k_i$	$\div \sqrt{2}$	Scaled score
V1 (Robot t0)	1.00	/ 1.4142	0.7071
V2 (Tablet t0)	0.50	/ 1.4142	0.3536
V3 (Robot t1)	1.25	/ 1.4142	0.8839
V4 (Tablet t1)	1.00	/ 1.4142	0.7071

4.2 แปลงเป็นน้ำหนักด้วย Softmax

$$\alpha_i = \frac{exp(score_i)}{\sum_j exp(score_j)}$$

น้ำหนักความสนใจ (Attention Weight)

Token	Scaled score	Softmax weight (α_i)
V1 (Robot t0)	0.7071	0.2567
V2 (Tablet t0)	0.3536	0.1803
V3 (Robot t1)	0.8839	0.3063
V4 (Tablet t1)	0.7071	0.2567

4.3 หาค่าผลรวมถ่วงน้ำหนัก (Context Vector)

$h_{V3} = \sum \alpha_i v_i$ (เพราะ $W_V = I \Rightarrow v_i$ = token vector)

$h_{V3} = [0.6914, 0.5901]$ (ตัวเลขจำลองเพื่อสาธิต)

5 ตีความผลลัพธ์ (Interpretation)

1. V3 ให้น้ำหนักสูงสุดกับตัวเอง (0.3063) เพราะมีความคล้ายกันสูงสุดกับตัวเอง ทั้งในเชิงพื้นที่และเวลา
2. เชื่อมโยงกับหุ่นยนต์ที่ t0 (0.2567) เพราะเป็นวัตถุชนิดเดียวกัน ช่วยให้เข้าใจการเคลื่อนไหวของหุ่นยนต์ข้ามเวลา
3. เชื่อมโยงกับแท็บเล็ตที่ t1 (0.2567) เพราะอยู่ในเฟรมเดียวกัน จึงจับบริบทของสิ่งแวดล้อมในพื้นที่เดียวกัน
4. ให้น้ำหนักน้อยสุดกับแท็บเล็ดที่ t0 (0.1803) เพราะไกลทั้งในเชิงพื้นที่และเวลา ความคล้ายจึงน้อยกว่า

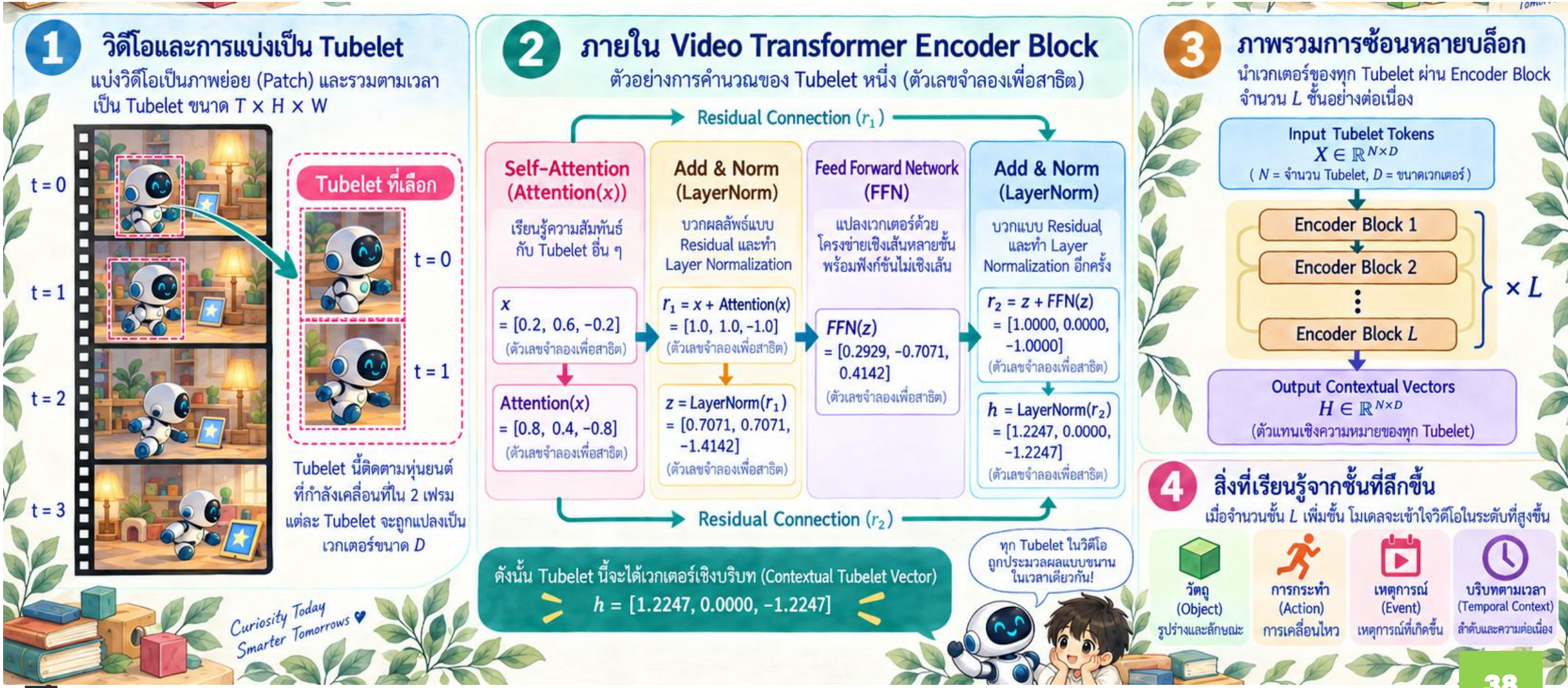
Self-Attention ช่วยให้โมเดลมองเห็นทั้ง "ใกล้ในภาพ" และ "ไกลข้ามเวลา" ในครั้งเดียว!

สรุปแนวคิดสำคัญ

- Video Transformer ใช้ Self-Attention เชื่อมพื้นที่และเวลา
- ในทางปฏิบัติ อาจแยกคำนวณ Spatial และ Temporal Attention หรือใช้แบบรวม (Joint Attention)
- ตัวเลขทั้งหมดในภาพนี้เป็น "ตัวเลขจำลองเพื่อสาธิต" เพื่ออธิบายแนวคิดเท่านั้น
- แนวคิดนี้ช่วยให้โมเดลเข้าใจภาพเคลื่อนไหว เหตุการณ์ และบริบทของวิดีโอได้ดีขึ้น

Small Ideas Big Possibilities

Video Transformation Encoder : Transformer Encoder Block



Video Transformation Encoder : Pooling

1 ตัวอย่างวิดีโอและ Tubelet Vectors

แบ่งวิดีโอเป็น Tubelet (พื้นที่ × เวลา) แล้วแปลงเป็นเวกเตอร์

t = 0 (ช่วงต้นคลิป)

Tubelet 1: หุ่นยนต์ (t0)



$h_1 = [0.5, 0.1, 0.2, 0.3]$

(ตัวเลขจำลองเพื่อสาธิต)

Tubelet 2: แท็บเล็ต (t0)



$h_2 = [0.2, 0.6, 0.1, 0.0]$

(ตัวเลขจำลองเพื่อสาธิต)

t = 1 (ช่วงปลายคลิป)

Tubelet 3: หุ่นยนต์ (t1)



$h_3 = [0.6, 0.2, 0.4, 0.1]$

(ตัวเลขจำลองเพื่อสาธิต)

Tubelet 4: แท็บเล็ต (t1)



$h_4 = [0.3, 0.7, 0.5, 0.2]$

(ตัวเลขจำลองเพื่อสาธิต)

h_i คือเวกเตอร์ของ Tubelet ขนาด 4 มิติ (ตัวเลขจำลองเพื่อสาธิต)

2 วิธี Pooling สู่ Video Vector

มีหลายวิธีในการรวมข้อมลจากหลาย Tubelet

2.1 Mean Pooling

เฉลี่ยทุก Tubelet อย่างเท่าเทียม

$$z_{mean} = \frac{1}{N}\sum_{i=1}^{N} h_i$$

เมื่อ $N = 4$

$z_{mean} = [0.40, 0.40, 0.30, 0.15]$

(ตัวเลขจำลองเพื่อสาธิต)

- ✓ เรียบง่าย ไม่ซับซ้อน
- ✓ ใช้ข้อมูลจากทุกส่วน
- ✗ อาจทำให้รายละเอียดเด่น ๆ ถูกเจือจาง

2.2 Max Pooling

เลือกค่ามากที่สุดในแต่ละมิติ

$$z_{max}[j] = \max_i(h_i[j])$$

สำหรับทุกมิติ j

$z_{max} = [0.6, 0.7, 0.5, 0.3]$

(ตัวเลขจำลองเพื่อสาธิต)

- ✓ เน้นสัญญาณเด่นที่สุด
- ✓ ทนทานต่อสัญญาณรบกวน
- ✗ อาจละเลยข้อมูลที่มีประโยชน์อื่น ๆ

2.3 [CLS] Token

ใช้เวกเตอร์เฉพาะที่เรียนรู้ให้สรุปทั้งคลิป

$$z_{[CLS]} = h_{cls}$$

(เรียนรู้จากข้อมูล)

$z_{[CLS]} = [0.4, 0.5, 0.3, 0.2]$

(ตัวเลขจำลองเพื่อสาธิต)

- ✓ ถูกฝึกให้สรุปภาพรวม
- ✓ มักได้ผลดีในการปฏิบัติ
- ✗ ขึ้นกับการฝึกโมเดล และอาจขาดรายละเอียดบางส่วน

2.4 Attention Pooling

ให้น้ำหนักแต่ละ Tubelet ตามความสำคัญ

$$z_{att} = \sum_{i=1}^{N} \alpha_i h_i$$

โดย $\sum \alpha_i = 1$

น้ำหนักความสำคัญ (α_i)

$\alpha_1 = 0.15$ (t0, หุ่นยนต์)
$\alpha_2 = 0.15$ (t0, แท็บเล็ต)
$\alpha_3 = 0.35$ (t1, หุ่นยนต์)
$\alpha_4 = 0.35$ (t1, แท็บเล็ต)

$\sum \alpha_i = 1.0$

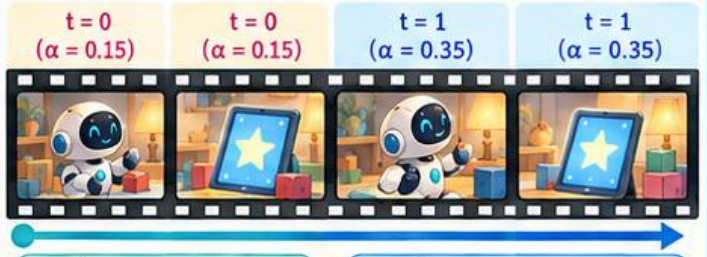
$z_{att} = [0.42, 0.42, 0.36, 0.15]$

(ตัวเลขจำลองเพื่อสาธิต)

- ✓ เน้นส่วนที่สำคัญ (เช่น ช่วงปลายคลิป)
- ✓ ยืดหยุ่น เรียนรู้น้ำหนักได้
- ✗ ซับซ้อนกว่าวิธีอื่น

3 น้ำหนักตามเวลา (Temporal Weighting)

ในตัวอย่างนี้ ช่วงปลายคลิป (t = 1) มีน้ำหนักรวมมากกว่า



น้ำหนักรวมช่วงต้นคลิป
0.15 + 0.15 = 0.30

น้ำหนักรวมช่วงปลายคลิป
0.35 + 0.35 = 0.70

ช่วยให้โมเดลให้ความสำคัญกับเหตุการณ์ที่เกิดขึ้นภายหลัง ซึ่งมักมีข้อมูลสำคัญกว่าในหลายสถานการณ์

4 สรุปข้อดีข้อจำกัดของแต่ละวิธี

วิธี	จุดเด่น	ข้อจำกัด	เหมาะกับงาน
Mean	เรียบง่าย ใช้ทุกส่วน	อาจเจือจางข้อมูลเด่น	งานทั่วไป
Max	ดึงสัญญาณเด่น	ละเลยข้อมูลอื่น	เหตุการณ์เด่น/ตรวจจับ
[CLS]	เรียนรู้ให้สรุปภาพรวม	ขึ้นกับการฝึกโมเดล	งานที่ใช้ Transformer
Attention	ยืดหยุ่น ให้ความสำคัญตามเนื้อหาและเวลา	ซับซ้อนกว่า	งานที่ต้องเข้าใจบริบทเชิงเวลา

5 ภาพรวมกระบวนการ: จาก Tubelet Vectors สู่การใช้งาน



วิดีโอ (หลายเฟรม)

→ แบ่งเป็น Tubelet Vectors



(ตัวเลขจำลองเพื่อสาธิต)

→ **Pooling** (Mean / Max / [CLS] / Attention)

→ z_{video} [0.42, 0.42, 0.36, 0.15] (ตัวเลขจำลองเพื่อสาธิต)

→ **Projection** (เช่น Linear Layer)

→ ใช้ในการค้นหาหรือผสานหลายสื่อ

- Similarity Search (ค้นหาวิดีโอที่คล้ายกัน)
- Multimodal Fusion (ผสมกับข้อความ/เสียง)

"Small representations, big understanding."

เทคโนโลยีที่เข้าใจมนุษย์ เพื่อการเรียนรู้ที่ดีกว่า

ตัวเข้ารหัสที่ฝึกล่วงหน้าและการจับคู่ภาพ–ข้อความด้วย CLIP

ภาพและข้อความเชื่อมโยงความหมายข้าม Modality สู่คำแนะนำที่ดีกว่า

Pretrained Image Encoder + Pretrained Text Encoder → Shared Embedding Space

เรียนรู้จากโลกกว้างเพื่อนำไปแนะนำสิ่งที่ใช่ให้กับทุกคน

หลาย Modality หลายมุมมอง สร้างประสบการณ์ที่ดีกว่าเสมอ

1 ข้อมูลคู่ภาพ–ข้อความ

ตัวอย่างข้อมูลแบบ mini-batch ที่มีทั้งคู่ตรงกันและคู่ไม่ตรงกัน

	ภาพ I_i	ข้อความ T_j	
I_1		T_1 “toys in a room”	คู่ตรงกัน (Positive)
I_2		T_2 “blue ocean”	คู่ตรงกัน (Positive)
I_3		T_3 “forest adventure”	คู่ตรงกัน (Positive)

ตัวอย่างคู่ไม่ตรงกัน (Negative)

- I_1 ✕ T_2 “blue ocean”
- I_2 ✕ T_3 “forest adventure”
- I_3 ✕ T_1 “toys in a room”

ในการฝึก จะใช้ทั้งคู่ตรงกันและคู่ไม่ตรงกันจำนวนมาก

2 Pretrained Dual Encoders

ใช้ตัวเข้ารหัสที่ฝึกล่วงหน้าสองตัว สำหรับภาพและข้อความ

Image I_i → Image Encoder (ViT / CNN) → h_i^I

Text T_j “toys in a room” → Text Encoder (Transformer) → h_j^T

Pretrained = เรียนรู้น้ำหนักจากข้อมูลคู่ภาพ–ข้อความขนาดใหญ่แล้ว ไม่ต้องเริ่มฝึกจากศูนย์

ทางเลือกในการถ่ายโอนความรู้ (Transfer Learning)

- **Freeze** ใช้น้ำหนักเดิม ไม่ฝึกเพิ่ม
- **Linear Probe** ฝึกเฉพาะชั้นสุดท้าย
- **Fine-tune / Adapter** ปรับน้ำหนักบางส่วนหรือใช้ Adapter

3 Projection + L^2 Normalize

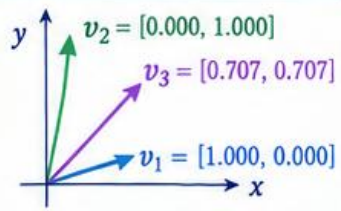
แปลงเป็นเวกเตอร์ในมิติเดียวกัน และทำให้มีความยาวเท่ากับ 1

$$v_i = \text{Normalize}\left(W^I h_i^I\right)$$

$$t_j = \text{Normalize}\left(W^T h_j^T\right)$$

$$\|v_i\|_2 = \|t_j\|_2 = 1$$

ภาพและข้อความอยู่ใน Shared Embedding Space เดียวกัน



ตัวอย่างเวกเตอร์ที่ถูก Normalize แล้ว (2 มิติ เพื่อการแสดงผล)

- v_1 = [1.000, 0.000]
- v_2 = [0.000, 1.000]
- v_3 = [0.707, 0.707]
- t_1 = [0.981, 0.196]
- t_2 = [0.100, 0.995]
- t_3 = [0.707, 0.707]

4 Similarity Matrix

คำนวณความคล้ายด้วย cosine similarity

$$\cos(v_i, t_j) = v_i^T t_j$$

Similarity Matrix (3 × 3)

	T_1	T_2	T_3
I_1	**0.981**	0.100	0.707
I_2	0.196	**0.995**	0.707
I_3	0.832	0.774	**1.000**

ค่าบนแนวทแยงสูง = คู่ภาพ–ข้อความตรงกัน



5 Contrastive Learning

ฝึกให้ภาพ–ข้อความที่ตรงกันใกล้กัน และที่ไม่ตรงกันไกลออกไป

Scaled logits

$$s_{ij} = \frac{v_i^T t_j}{\tau}, \quad \tau = 0.07$$

Symmetric contrastive loss

$$\mathcal{L}_{CLIP} = \frac{\mathcal{L}_{image \to text} + \mathcal{L}_{text \to image}}{2}$$

ดึง Positive pairs ให้ใกล้กัน

ผลัก Negative pairs ให้ออกจากกัน

คำนวณแบบสองทิศทาง

- **Row-wise softmax (Image → Text)** สำหรับแต่ละภาพ กระจายความน่าจะเป็นไปยังข้อความทั้งหมด
- **Column-wise softmax (Text → Image)** สำหรับแต่ละข้อความ กระจายความน่าจะเป็นไปยังภาพทั้งหมด

6 Matching และ Retrieval

ใช้ความคล้ายเพื่อค้นหาข้อความหรือภาพที่ตรงกัน

Image Query: I_1



ค้นหาข้อความที่ตรงกันจากรายการทั้งหมด

Rank	ข้อความ (Text Candidate)	cosine
1	“toys in a room”	0.981
2	“forest adventure”	0.707
3	“blue ocean”	0.100

Text Query: “forest adventure” ค้นหาภาพที่ตรงกัน



Zero-shot matching: เปรียบเทียบคำบรรยายใหม่ได้ โดยไม่ต้องฝึก classifier ใหม่

การใช้ CLIP ใน Multimodal Recommender System

Item Image + Title/Synopsis (หลาย Modality) → CLIP Encoders (Image + Text) → Aligned Item Vectors (ใน Shared Space) → User Profile (จากประวัติการใช้งาน) → Cosine Similarity (เปรียบเทียบความคล้าย) → Ranked Recommendations

ตัวอย่างการใช้งาน

- ค้นหารายการด้วยข้อความ
- แนะนำจากความคล้ายภาพ–ข้อความ
- ช่วย Item Cold Start

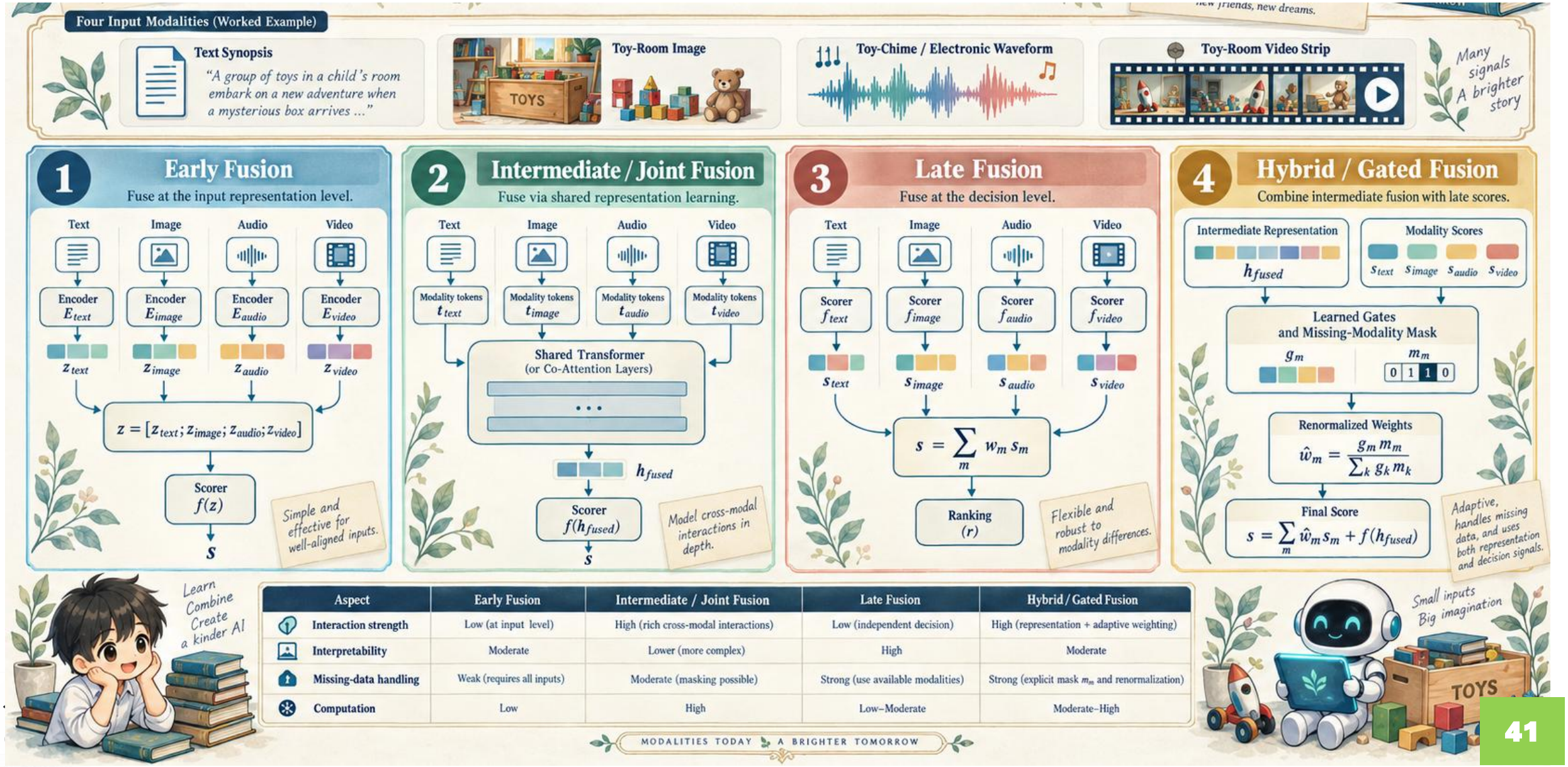
CLIP จับความสัมพันธ์เชิงความหมายข้าม Modality แต่ควรตรวจ domain shift, bias และปรับ fine-tune เมื่อข้อมูลเฉพาะทาง

เข้าใจหลาย Modality แนะนำได้ตรงใจยิ่งขึ้น

สรุปขั้นตอน: Paired Data → Dual Encoders → Projection → Normalize → Similarity → Contrastive Loss → Retrieval

จากการเรียนรู้ข้ามภาพและข้อความ สู่ระบบแนะนำที่ใช้งานได้จริง

Mutimodal Fusion Patterns



Aspect	Early Fusion	Intermediate / Joint Fusion	Late Fusion	Hybrid / Gated Fusion
Interaction strength	Low (at input level)	High (rich cross-modal interactions)	Low (independent decision)	High (representation + adaptive weighting)
Interpretability	Moderate	Lower (more complex)	High	Moderate
Missing-data handling	Weak (requires all inputs)	Moderate (masking possible)	Strong (use available modalities)	Strong (explicit mask m_m and renormalization)
Computation	Low	High	Low–Moderate	Moderate–High



Multimodal User Profile Construction

Text • Image • Audio • Video → One Personalized Representation

MovieMini
i11 Toy Story 5 (2026)





1 MULTIMODAL HISTORY

User u3 interacts with items





i11 Toy Story 5 (2026)

★★★★☆ 4.5



i07 Ocean Friends

★★★★☆ 4.0



i23 Forest Adventure

★★☆☆☆ 2.0



i35 Space Journey

★★★★★ 5.0

• • • more interactions • • •

2 PER-MODALITY ITEM EMBEDDINGS

same item, four semantic views



i11 Toy Story 5 (2026)

Text → Text Transformer → $v_i^{text} \in \mathbb{R}^d$ [0.28 -0.12 0.51 …]

Image → Image / Patch Transformer → $v_i^{image} \in \mathbb{R}^d$ [0.33 0.41 -0.08 …]

Audio → Audio Transformer → $v_i^{audio} \in \mathbb{R}^d$ [-0.21 0.37 0.11 …]

Video → Video Transformer → $v_i^{video} \in \mathbb{R}^d$ [0.19 -0.04 0.62 …]

The same item, four complementary views

3 PER-MODALITY USER PROFILES

Aggregate user history per modality

$$u_m = \text{Normalize}\left(\sum_i w_{ui} \cdot v_i^m\right)$$

- Positive (like, high rating) $w_{ui} > 0$
- Negative (dislike, low rating) $w_{ui} < 0$
- Rating-centered weighting $w_{ui} = (r_{ui} - \bar{r}_u)$

Text profile u_{text} [0.42 0.11 -0.17 …] $d = 64$

Image profile u_{image} [0.36 0.48 0.03 …] $d = 64$

Audio profile u_{audio} [-0.05 0.52 0.28 …] $d = 64$

Video profile u_{video} [0.21 0.10 0.62 …] $d = 64$



Each profile captures what you like in a different way!

4 PROFILE FUSION

Combine per-modality profiles

A Early profile fusion

Concatenate and normalize

$$u = \text{Normalize}\left(\begin{bmatrix} \sqrt{\alpha_t}\, u_{text} \\ \sqrt{\alpha_i}\, u_{image} \\ \sqrt{\alpha_a}\, u_{audio} \\ \sqrt{\alpha_v}\, u_{video} \end{bmatrix}\right) \quad u \in \mathbb{R}^{4d}$$

B Late score fusion

Fuse similarity scores at prediction time

$$s(u,j) = \sum_m \alpha_m \cdot \cos(u_m, v_j^m)$$

Each modality votes!

C Gated / quality-aware fusion

Adaptive weights per user and item

$$\alpha_m(u,j) = \frac{\exp(g_m)}{\sum_{k \in M} \exp(g_k)}$$

g_m depends on:
- user preference
- modality availability
- signal quality
- item characteristics

Optional joint profile (cross-attention)

u_m (m=1..4) → Cross-Attention → u_{joint}

Capture cross-modal interactions (e.g., text + image alignment)

5 ROBUST RANKING

Score candidates and recommend

Candidate: i11 Toy Story 5 (2026)



Per-modality similarity (cosine)

- $s_{text} = 0.73$
- $s_{image} = 0.68$
- $s_{audio} = 0.41$
- $s_{video} = 0.62$

Weighted final score ($\alpha_t = 0.35$, $\alpha_i = 0.30$, $\alpha_a = 0.10$, $\alpha_v = 0.25$)

$s(u, i11) = 0.64$



Because your text interests and visual style both match.

Missing modality example

Audio unavailable for this item

→ mask audio, renormalize remaining α (do not fill with zero).

Original α: [0.35, 0.30, 0.10, 0.25]

Renormalized (T, I, V): [0.41, 0.35, 0.24]

Top-N Recommendation (for u3)

1	i11 Toy Story 5 (2026)	0.64
2	i35 Space Journey	0.58
3	i07 Ocean Friends	0.52

… more items …

Best Practices & Evaluation Checklist

- ☑ L2 normalize each space ($\|v\|_2 = 1, \|u\|_2 = 1$)
- ☑ Exclude seen items during recommendation
- ☑ Prevent leakage (time-aware split)
- ☑ Log modality masks (availability & quality)
- ☑ Evaluate ablations (single modality / fusion variants)

Different signals A more complete you



Leave-One-Out Evaluation Protocol

Hold out one known liked item per user, rank among unseen items, and average the results.



1 Choose one eligible user

A user with at least one liked item (e.g., from MovieMini-12).



Example: user u3

2 Hide exactly one liked item

Randomly select one liked item as the ground truth and hide it from training.



i11
Toy Story 5 (2026)

Hidden test item (ground truth)

3 Build the user profile from training

Use only the remaining liked items (exclude the hidden item).

Training items for u3
(e.g., liked, except i11)



No test leakage

Do not use the hidden item i11 in the user profile.

4 Form candidate items

Create candidates from unseen items, including the hidden item i11, but excluding training (rated) items.

Candidate set for u3



Unseen items only
(exclude training items)

5 Score and rank all candidates

Compute a relevance score for each candidate and rank in descending order. Find the rank r of the hidden item i11.

Rank	Item	Score	
1	i4	0.83	
2	i1	0.62	
3	i11	0.58	r = 3
4	i7	0.41	
5	i9	0.37	
…			

6 Compute metrics (for K = 3)

Using the rank r of the hidden item i11, compute:

$$\text{Hit@3} = \mathbf{1}[r \le 3]$$

$$\text{RR} = \frac{1}{r}$$

$$\text{nDCG@3} = \frac{1}{\log_2(r+1)} \quad \text{if } r \le 3 \text{ else } 0$$

Example (r = 3)
Hit@3 = 1
RR = 1/3
$\text{nDCG@3} = \frac{1}{\log_2(4)} = 0.5$

7 Repeat for every eligible liked item and user

For each user, repeat Steps 2–6 for every liked item (one left out at a time).
In MovieMini-12 (2024–2026), user u3 has 20 liked items ⇒ 20 leave-one-out cases.



Average the metrics

Compute the mean Hit@3, RR, and nDCG@3 over all 20 cases (and all eligible users, if evaluating globally).



Worked Example (MovieMini-12, 2024–2026)

User: u3
Held-out (test) item: i11 · Toy Story 5 (2026)
Training items: other liked items of u3 (exclude i11)
Candidates: unseen items including i11 (exclude training items)
Result: hidden item i11 is at rank r = 3
Metrics: Hit@3 = 1, RR = $\frac{1}{3}$, nDCG@3 = 0.5

One by one.
A fair test
for a brighter
tomorrow.

